

Pandas: Labeled Data Structures for Real-World Data

Prepared By

Dr. Islam Zakaria

Lecturer at the Information Technology Department, FCAIH, Capital University

The DataFrame Architecture & Data Engineering

Introduction to Pandas

□ Definition

- Pandas (“Panel Data”) is a high-performance, open-source data analysis library built on top of NumPy.
- It introduces flexible data structures designed specifically for relational, labeled, and heterogeneous datasets, such as spreadsheets, CSV files, database tables, and time series.

Relational datasets

- ❑ **Meaning:** Data organized in rows and columns, where each row is a record and each column is a variable (attribute).
- ❑ **Key idea:** Each column has a single, consistent data type, and relationships can exist between tables via keys.
- ❑ **Examples:** Excel spreadsheets, CSV files, Database tables (SQL).

Student_ID	Name	GPA
101	Ali	3.4
102	Sara	3.9

Labeled datasets

- ❑ **Meaning:** Data that includes a label or target value for each record.
- ❑ **Key idea:** Used mainly in supervised machine learning.

- ❑ **Examples:**

- A dataset where the final column is a class or outcome
- Time series with labeled events

Age	Income	Buy_Product
25	3000	No
40	7000	Yes

- ❑ **Here:**

- **Age and Income** → **features**
- **Buy_Product** → **label (target/output)**

Heterogeneous datasets

- ❑ **Meaning:** Data containing different types of data in the same dataset.
- ❑ **Key idea:** Not all columns have the same data type or structure.
- ❑ **Examples of mixed data types:**
 - Numerical (age, salary)
 - Categorical (gender, country)
 - Text (comments) Dates/time
 - Images or signals (in advanced cases)

User_ID	Age	Gender	Join_Date	Feedback
1	30	Male	2023-01-10	"Very good"

- ❑ **This dataset is heterogeneous because it mixes:** Numbers, Categories, Dates and Text.

Time series datasets

- ❑ **Meaning:** Data collected over time, where the order of observations matters.
- ❑ **Key idea:** Each data point is associated with a timestamp.
- ❑ **Examples:**
 - Stock prices
 - Temperature readings
 - Network traffic over time (important in IoV / networking research)

Time	Temperature
10:00 AM	22.5
10:05 AM	22.7
10:10 AM	23.0

Putting it all together

❑ When someone says:

- “relational, labeled, and heterogeneous datasets, such as spreadsheets, CSV files, database tables, and time series”

❑ They mean:

- **Relational** → structured in tables (rows & columns)
- **Labeled** → includes target/output values
- **Heterogeneous** → contains multiple data types
- **Examples** → spreadsheets, CSVs, databases, and time-based data

Pandas - Core Architecture

1. **Series (1D).** A Series is a labeled one-dimensional array in pandas.

➤ **Key characteristics**

- Unlike NumPy, a Pandas Series is a one-dimensional labeled array that can store multiple data types simultaneously (integers, strings, floats, etc.).
- Pandas does not actually store them as separate types. Instead, it **upcasts** the entire Series to the object dtype.
- The series is not stored in a contiguous memory block; it is stored as a collection of pointers to Python objects.

➤ **Similar to:** **list** (ordered values), a **dictionary** (key–value pairs)

➤ **Internal structure:** A Series consists of two main components:

- **Values** → stored internally as a NumPy array
- **Index** → labels identifying each value

NumPy vs. Pandas

Feature	NumPy Array	Pandas Series
Data Type	Homogeneous (fixed)	Heterogeneous (via object)
Memory	Contiguous / Highly Efficient	Less efficient for mixed types
Performance	Fast (Vectorized)	Slower for object types
Indexing	Integer-based only	Label-based + Integer-based

Example

```
import pandas as pd

# Creation from list with default index (0, 1, 2...)
s = pd.Series([10, 20, 30, 40])
# Output: 0    10
#         1    20
#         2    30
#         3    40
#         dtype: int64

# Creation with custom index
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
# Output: a    10
#         b    20
#         c    30
#         d    40
#         dtype: int64
```

Pandas - Core Architecture (Cont.)

2. **DataFrame (2-Dimensional)**. A **DataFrame** is a **two-dimensional labeled data structure** with rows and columns.

➤ **Comparable to**

- A spreadsheet (Excel)
- An SQL table
- A collection (dictionary) of aligned Series objects

➤ **Key idea**

- Each column is a Series (a column *can* store multiple different Python types)
- All columns share the same index (row labels)

Example

```
python

df = pd.DataFrame({
    "Name": ["Ali", "Sara", "Omar"],
    "GPA": [3.4, 3.9, 3.2]
})
```

Index	Name	GPA
0	Ali	3.4
1	Sara	3.9
2	Omar	3.2

- **df["Name"]** → a Series
- **df["GPA"]** → a Series

Example (Cont.)

❑ Why shared Index matters?

➤ **Because columns share the same index, pandas enables:**

- **Column-wise operations:** `df["GPA"] + 0.1`
- **Easy filtering:** `df[df["GPA"] > 3.5]`
- **Grouping and aggregation:** `df.groupby("Name").mean()`
- **Fast vectorized arithmetic:** Operations are performed **element-by-element** without explicit loops (much faster than pure Python).

Comparison Table

A Series is a labeled one-dimensional array, while a DataFrame is a two-dimensional table composed of aligned Series objects sharing the same index.

Feature	Series	DataFrame
Dimensions	1D	2D
Structure	Values + Index	Rows + Columns
Similar to	List / Dict	Table / Spreadsheet
Columns	N/A	Series objects

The Alignment Paradigm

❑ Label-Based Alignment

- Unlike NumPy arrays (which align strictly by position), Pandas aligns data by label (**Index** and **column names**).
- **When performing arithmetic, join operations, or merges:**
 - Values are matched by their labels, not just their order.
 - Missing labels are automatically filled with **NaN** (missing data marker).

❑ Key Insight

- **Pandas extends NumPy from numerical arrays to labeled, real-world data.**
- **Its Series and DataFrame structures, combined with automatic label alignment, make it the foundation of modern data analysis in Python.**

The Alignment Paradigm (Cont.)

□ Why This Matters

➤ **Handles real-world datasets where:**

- Rows may not line up
- Columns may be missing
- Different data sources need merging

➤ **Built-in alignment ensures:**

- Safe arithmetic
- Automatic missing-data propagation
- Reduced risk of mixing up mismatched rows

NumPy Alignment vs Pandas Alignment

Feature	NumPy Alignment	Pandas Alignment
Logic	Positional (Index 0 to Index 0)	Label-based ('Apples' to 'Apples')
Size Constraint	Shapes must match (or broadcast)	Shapes can be completely different
Missing Data	Crashes or requires manual padding	Automatically handles via "Outer Join" logic

Example 1 - NumPy's Position Trap

```
import numpy as np

arr_A = np.array([100, 200, 300, 400]) # Q1, Q2, Q3, Q4 revenue
arr_B = np.array([50, 80, 120])         # Missing Q4 data!

# This will error or give wrong results
try:
    arr_A + arr_B # Misalignment disaster!
except ValueError as e:
    print(f"NumPy Error: {e}") # Different lengths
```

Example 2 - Pandas' Label Safety

```
import pandas as pd

rev = pd.Series([100, 200, 300, 400],
                index=['Q1', 'Q2', 'Q3', 'Q4'])
costs = pd.Series([50, 80, 120],
                  index=['Q1', 'Q2', 'Q3'])

profit = rev - costs
print(profit)

# Q1      50.0  # 100-50
# Q2     120.0  # 200-80
# Q3     180.0  # 300-120
# Q4       NaN  # costs missing for Q4 → obvious!
```

Example 3 - Real-World Merge Scenarios

```
# Sales data from Store A
store_A = pd.DataFrame({
    'Product': ['Apple', 'Banana', 'Cherry'],
    'Price': [1.2, 0.5, 3.0]
})

# Sales data from Store B (different products, different order)
store_B = pd.DataFrame({
    'Product': ['Banana', 'Cherry', 'Date', 'Elderberry'],
    'Price': [0.6, 2.8, 4.0, 2.5]
})

# Compare prices safely
comparison = store_A.set_index('Product')['Price'] - store_B.set_index('Product')['Price']
print("Price differences (A - B):")
print(comparison)

# Apple      NaN (only in A)
# Banana    -0.1
# Cherry      0.2
# Date       NaN (only in B)
# Elderberry NaN (only in B)
```

set_index() is a Pandas DataFrame method that moves one or more columns into the index, making them the row labels instead of regular data columns. This makes operations like lookups, joins, and alignment semantic, not positional.

Example 4 - Duplicates in Both DataFrames (Problem)

```
import pandas as pd
import numpy as np

# Store A has duplicate 'Banana' entries (maybe different types?)
store_A = pd.DataFrame({
    'Product': ['Apple', 'Banana', 'Banana', 'Cherry'],
    'Price': [1.2, 0.5, 0.8, 3.0] # Two different banana prices!
})

# Store B also has duplicate 'Cherry'
store_B = pd.DataFrame({
    'Product': ['Banana', 'Cherry', 'Cherry', 'Date'],
    'Price': [0.6, 2.8, 3.2, 4.0]
})

print("Store A (with duplicate Banana):")
print(store_A)
print("\nStore B (with duplicate Cherry):")
print(store_B)

# What happens when we set_index?
try:
    comparison = store_A.set_index('Product')['Price'] - store_B.set_index('Product')['Price']
    print("\nAttempting comparison...")
    print(comparison)
except Exception as e:
    print(f"\nError: {type(e).__name__}: {e}")
```

Output:

Store A (with duplicate Banana):

	Product	Price
0	Apple	1.2
1	Banana	0.5
2	Banana	0.8
3	Cherry	3.0

Store B (with duplicate Cherry):

	Product	Price
0	Banana	0.6
1	Cherry	2.8
2	Cherry	3.2
3	Date	4.0

Error: ValueError: cannot reindex on an axis with duplicate labels

When labels are duplicated, Pandas doesn't know: **Which Banana in Store A (0.5 or 0.8) to match with which Banana in Store B (0.6)?**

Example 4 - Duplicates in Both DataFrames (Solution)

```
# Aggregate duplicates before comparison
store_A_agg = store_A.groupby('Product')['Price'].mean() # or .first(), .min(), etc.
store_B_agg = store_B.groupby('Product')['Price'].mean()

print("Store A (aggregated by mean):")
print(store_A_agg)
print("\nStore B (aggregated by mean):")
print(store_B_agg)

comparison = store_A_agg - store_B_agg
print("\nPrice difference (A mean - B mean):")
print(comparison)
```

```
Store A (aggregated by mean):
Product
Apple      1.2
Banana     0.65 # (0.5 + 0.8) / 2
Cherry     3.0
Name: Price, dtype: float64
```

```
Store B (aggregated by mean):
Product
Banana     0.6
Cherry     3.0 # (2.8 + 3.2) / 2
Date       4.0
Name: Price, dtype: float64
```

```
Price difference (A mean - B mean):
Product
Apple      NaN
Banana     0.05
Cherry     0.0
Date       NaN
dtype: float64
```

Example 5 - Time Series Alignment (Critical!)

```
import numpy as np

# Stock prices with different timestamps
stock_A = pd.Series(
    [100, 101, 102, 103],
    index=pd.to_datetime(['2023-01-01', '2023-01-02',
                          '2023-01-03', '2023-01-04'])
)

stock_B = pd.Series(
    [200, 201, 202],
    index=pd.to_datetime(['2023-01-02', '2023-01-03',
                          '2023-01-05']) # Different days!
)

# Pandas aligns by date automatically
combined = pd.DataFrame({'Stock_A': stock_A, 'Stock_B': stock_B})
print(combined)
```

#		Stock_A	Stock_B	
#	2023-01-01	100.0	NaN	
#	2023-01-02	101.0	200.0	# Aligned!
#	2023-01-03	102.0	201.0	# Aligned!
#	2023-01-04	103.0	NaN	
#	2023-01-05	NaN	202.0	

Example 5 - Business Impact

```
# Monthly sales targets vs actuals (different months)
targets = pd.Series({
    'Jan': 10000,
    'Feb': 12000,
    'Mar': 15000,
    'Apr': 18000 # April target set
})

actuals = pd.Series({
    'Jan': 9500,
    'Feb': 12500,
    'Mar': 14000
    # April actuals not in yet
})

# Automatic alignment prevents mismatching months
performance = actuals - targets
print("Monthly Performance (Actual - Target):")
print(performance)
# Jan    -500.0
# Feb     500.0
# Mar   -1000.0
# Apr      NaN # Clearly shows April data missing
```

Data (I/O) in Pandas

❑ The Reader Family (pd.read_*)

- **read_csv(): The workhorse.** Key parameters: **sep**, **header**, **index_col**, **parse_dates**.
- **read_excel():** Requires **openpyxl**. Handles multiple sheets.
- **read_sql():** Bridges Python with databases (PostgreSQL, SQLite) using SQLAlchemy.

❑ The Writer Family (.to_*):

- **to_csv(): Standard export.** **Tip:** Always use **index=False** **unless the index contains meaningful data (like Dates)**.
- **to_parquet(): (Advanced) Binary format for massive datasets; preserves schema better than CSV.**

The Reader Family - read_csv()

The most commonly used pandas function for loading external data into a DataFrame.

Why it is important

- Works with CSV, TSV, and delimited text files.
- Highly flexible and efficient.
- Used in almost every data-science pipeline.

Key parameters

- **Sep**
 - **Defines** the column delimiter.
 - **Examples:** `' , ' ; ' \t '`

❑ Header

Specifies which row contains column names.

- **Header = 0** → first row (default).
- **Header = None** → no header.

❑ index_col

- Sets one column as the DataFrame index.
- Useful for IDs or timestamps.

❑ parse_dates

- Automatically converts columns to datetime objects.
- Essential for time-series analysis.

```
pd.read_csv("data.csv", sep=";", index_col="id", parse_dates=["date"])
```

The Reader Family - read_excel()

- ❑ **Used to load Excel files into pandas.**
- ❑ **Technical note**
 - **Requires the `openpyxl` engine for .xlsx files**
- ❑ **Supported features**
 - **Reading multiple sheets**
 - **Selecting specific sheets**
 - **Reading specific rows and columns**
 - **Handling mixed-type data**

```
pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

The Reader Family - read_sql()

- ❑ Reads data directly from SQL databases into a DataFrame.
- ❑ Supported databases
 - PostgreSQL
 - MySQL
 - SQLite
 - Others (via SQLAlchemy)
- ❑ Why it is ideal
 - Queries tables without exporting CSV files
 - Filters data at the database level
 - Efficient for large datasets

```
pd.read_sql("SELECT * FROM students WHERE gpa > 3.5", connection)
```

The Writer Family - to_csv()

- ❑ **The standard tool for exporting tabular data.**

- ❑ **Best practice**

 - Use **index=False** unless the index is meaningful (e.g., dates, unique identifiers).

- ❑ **Why it matters**

 - Produces clean, portable files
 - Compatible with Excel, R, databases, and BI tools

```
df.to_csv("output.csv", index=False)
```

The Writer Family - to_parquet()

❑ **A modern, column-oriented binary format designed for big data analytics.**

❑ **Key advantages**

- Preserves schema and data types
- Highly compressed
- Much faster than CSV for large datasets
- Optimized for cloud storage and analytics engines

❑ **Ideal use cases**

- Large-scale analytics
- Cloud data pipelines
- Machine-learning workflows

```
df.to_parquet("data.parquet")
```

CSV vs Parquet (Quick Comparison)

Feature	CSV	Parquet
Format	Text	Binary
File size	Large	Small
Schema	Not preserved	Preserved
Speed	Slow	Fast
Big Data	Not useful	Useful

Example 1 - CSV - Most Common Format

```
import pandas as pd

# Create sample data
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['NYC', 'LA', 'Chicago']
})

# Write to CSV
df.to_csv('data.csv', index=False)

# Read from CSV
df_read = pd.read_csv('data.csv')
print(df_read)
```

#	Name	Age	City
# 0	Alice	25	NYC
# 1	Bob	30	LA
# 2	Charlie	35	Chicago

Example 2 - Excel (.xlsx, .xls)

```
# Write to Excel
df.to_excel('data.xlsx', sheet_name='Sheet1', index=False)

# Read from Excel
df_read = pd.read_excel('data.xlsx', sheet_name='Sheet1')
print(df_read)
```

#	Name	Age	City
# 0	Alice	25	NYC
# 1	Bob	30	LA
# 2	Charlie	35	Chicago

Example 3 - JSON

```
# Write to JSON
df.to_json('data.json', orient='records', indent=2)

# Read from JSON
df_read = pd.read_json('data.json')
print(df_read)
```

#	Name	Age	City
# 0	Alice	25	NYC
# 1	Bob	30	LA
# 2	Charlie	35	Chicago

❑ **orient='records' means:**

- **Each** row of the DataFrame becomes a JSON object (dictionary).
- All rows are stored inside a list.

```
[
  {"col1": value1, "col2": value2, ...},
  {"col1": value3, "col2": value4, ...},
  ...
]
```

- ### ❑ **indent=2.** This makes the JSON readable by adding spacing and line breaks. Instead of one long line

```
[
  {
    "name": "Alice",
    "age": 25
  },
  {
    "name": "Bob",
    "age": 30
  }
]
```

Example 4 - Compressed Files

```
# Write compressed CSV
df.to_csv('data.csv.gz', index=False, compression='gzip')

# Read compressed CSV
df_read = pd.read_csv('data.csv.gz', compression='gzip')
print(df_read)
```

#	Name	Age	City
# 0	Alice	25	NYC
# 1	Bob	30	LA
# 2	Charlie	35	Chicago

Inspection & Exploratory Analysis

Structural Inspection

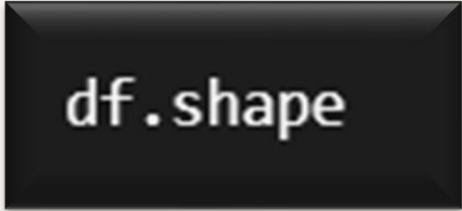
1. `.shape`

- Returns the dimensions of the DataFrame:

(Rows, Columns)

- **Purpose:**

- Quick check on dataset size
- Confirms successful data loading



`df.shape`

Inspection & Exploratory Analysis

2. `.info()`

One of the most important diagnostic methods in pandas.

➤ **Displays:**

- Column names
- Data types (dtypes)
- Non-null value counts (missing data detection)
- Memory usage



```
df.info()
```

➤ **Crucial for identifying:**

- Numeric or datetime columns incorrectly stored as object
- Columns containing missing values
- Opportunities for memory optimization (downcasting)

Inspection & Exploratory Analysis

3. `.columns`

Returns an Index object containing all column labels.

- The Index object is an **immutable, hashable array** that serves as the label-based address system for identifying and organizing rows and columns.
- Its primary advantages are lightning-fast data retrieval (**$O(1)$ lookups**) and automatic data alignment, which keeps related values synchronized during operations or merges.

➤ **Useful for:**

- Iteration over columns
- Renaming variables
- Feature selection

```
df.columns
```

Inspection & Exploratory Analysis

Content Inspection

1. `.head(n)` / `.tail(n)`

Displays the first or last n rows of the DataFrame.

➤ **Excellent for:**

- Quick data validation
- Detecting formatting or parsing errors
- Verifying correct data ingestion

```
df.head(5)  
df.tail(5)
```


Inspection & Exploratory Analysis

2. `.describe()`

Generates descriptive statistics for numeric columns.

➤ **Includes:**

- Mean and standard deviation
- Minimum and maximum values
- Quartiles (25%, 50%, 75%)

➤ **Provides a fast overview of:**

- Central tendency
- Variability
- Outliers
- Data distribution and spread

```
df.describe()
```

Example 1 - Structural Inspection

```
import pandas as pd
import numpy as np

# Create sample DataFrame with mixed data types
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'Age': [25, 30, 35, 28, np.nan],
    'Salary': [50000, 60000, 75000, 55000, 48000],
    'Department': ['HR', 'IT', 'IT', 'Finance', 'HR'],
    'Start_Date': pd.to_datetime(['2020-01-15', '2019-03-22',
                                   '2021-06-10', '2020-11-05', '2022-02-28']),
    'Bonus': [True, False, True, False, True]
})

### .shape - Dataset dimensions
print(f"DataFrame shape: {df.shape}")
# DataFrame shape: (5, 6)
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
# Rows: 5, Columns: 6

### .info() - Comprehensive metadata
print("\n=== DataFrame Info ===")
df.info()
```

```
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 5 entries, 0 to 4
# Data columns (total 6 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   Name        5 non-null      object
# 1   Age         4 non-null      float64
# 2   Salary      5 non-null      int64
# 3   Department  5 non-null      object
# 4   Start_Date  5 non-null      datetime64[ns]
# 5   Bonus       5 non-null      bool
# dtypes: bool(1), datetime64[ns](1), float64(1), int64(1), object(2)
# memory usage: 356.0+ bytes

### .columns - All column names
print(f"\nColumn names: {df.columns.tolist()}")
# Column names: ['Name', 'Age', 'Salary', 'Department', 'Start_Date', 'Bonus']

print(f"\nColumn index type: {type(df.columns)}")
# Column index type: <class 'pandas.core.indexes.base.Index'>
```

Example 2 - Content Inspection

```
print("First 3 rows:")
print(df.head(3))
#      Name  Age  Salary Department Start_Date  Bonus
# 0   Alice  25.0   50000         HR 2020-01-15   True
# 1    Bob   30.0   60000         IT 2019-03-22  False
# 2  Charlie  35.0   75000         IT 2021-06-10   True

print("\nLast 2 rows:")
print(df.tail(2))
#      Name  Age  Salary Department Start_Date  Bonus
# 3   Diana  28.0   55000    Finance 2020-11-05  False
# 4    Eve   NaN   48000         HR 2022-02-28   True

### .describe() - Statistical summary
print("\n=== Basic Statistics ===")
print(df.describe())
#      Age      Salary
# count  4.000000    5.000000
# mean   29.500000  57600.000000
# std    4.203173  10652.548203
# min    25.000000  48000.000000
# 25%    27.250000  50000.000000
# 50%    29.000000  55000.000000
# 75%    31.250000  60000.000000
# max    35.000000  75000.000000
```

```
# Include all columns (including non-numeric)
```

```
print("\n=== Statistics (include all) ===")
```

```
print(df.describe(include='all'))
```

```
#      Name  Age      Salary Department      Start_Date  Bonus
# count      5  4.0      5.000000         5              5      5
# unique      5  NaN          NaN         3            NaN      2
# top   Alice  NaN          NaN         IT            NaN     True
# freq      1  NaN          NaN         2            NaN      3
```

“**unique**”, “**top**”, and “**freq**” are typically reserved for categorical data.

Example 3 - Advanced Inspection

```
### Check for missing values
print("Missing values per column:")
print(df.isnull().sum())
# Name          0
# Age           1
# Salary        0
# Department     0
# Start_Date    0
# Bonus         0
# dtype: int64

### Check data types
print("\nData types:")
print(df.dtypes)
# Name          object
# Age           float64
# Salary        int64
# Department     object
# Start_Date    datetime64[ns]
# Bonus         bool
# dtype: object

### Memory usage (detailed)
print("\nMemory usage per column (bytes):")
print(df.memory_usage(deep=True))
# Index        128
# Name         331
# Age          40
# Salary       40
# Department   340
# Start_Date   40
# Bonus        5
# dtype: int64

### Unique values
print(f"\nUnique departments: {df['Department'].unique()}")
# Unique departments: ['HR' 'IT' 'Finance']

print(f"Number of unique names: {df['Name'].nunique()}")
# Number of unique names: 5

### Value counts
print("\nDepartment distribution:")
print(df['Department'].value_counts())
# IT          2
# HR          2
# Finance     1
# Name: Department, dtype: int64

print("\nBonus distribution:")
print(df['Bonus'].value_counts(dropna=False))
# True        3
# False       2
# Name: Bonus, dtype: int64
```

Example 4 - Quick Tips & Common Patterns

```
### Get column names by data type
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
print(f"Numeric columns: {numeric_cols}")
# Numeric columns: ['Age', 'Salary']

categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
print(f"Categorical columns: {categorical_cols}")
# Categorical columns: ['Name', 'Department']

### Check for duplicate rows
print(f"Duplicate rows: {df.duplicated().sum()}")
# Duplicate rows: 0
```

Summary Cheat Sheet

```
# Quick reference:
# df.shape          → (rows, columns)
# df.info()         → Data types, non-null counts, memory
# df.columns        → List of column names
# df.head(n)        → First n rows
# df.tail(n)        → Last n rows
# df.describe()     → Statistics for numeric columns
# df.dtypes         → Data type per column
# df.isnull().sum() → Count missing values per column
# df.memory_usage() → Memory usage per column
# df.nunique()       → Count unique values per column
# df.value_counts() → Frequency of values
```

Data Cleaning (The 80% Workload)

In real-world data science, most time is spent cleaning data, not modeling.

1. Handling Missing Data (NaN)

- **Detection:** Use Boolean masks to locate missing or valid values.

```
df.isna()  
df.notna()
```

- Returns **True** where values are **missing (NaN)**
- Essential for diagnostics before cleaning

Data Cleaning (Cont.)

2. Deletion

Remove rows or columns containing missing values.

```
df.dropna(axis=0, how='any')    # Drop rows with any NaN  
df.dropna(axis=1, how='all')    # Drop columns with all NaN
```

- **Axis = 0** → **rows**
- **Axis = 1** → **columns**
- Use carefully to avoid excessive data loss

Data Cleaning (Cont.)

3. Imputation

Replace missing values with constants or statistics.

```
df.fillna(0)
df.fillna(df.mean(numeric_only=True))
```

➤ Common strategies:

- **Constant values (e.g., 0, "Unknown")**

- **numeric_only = True:** This computes the mean **only** for each numeric column in df.

- **Statistical measures (mean, median, mode)**

➤ Preserves dataset size

Data Cleaning (Cont.)

Column Management

1. Renaming Columns

Improves readability and consistency.

```
df.rename(columns={'old': 'new'})
```

Data Cleaning (Cont.)

2. Type Casting

Convert columns to appropriate data types.

```
df['price'] = df['price'].astype(float)
```

➤ **Critical for:**

- Numerical computation
- Correct sorting and filtering
- Memory optimization

Example 1 - Creating a Messy Dataset

```
import pandas as pd
import numpy as np

# Create a realistic messy dataset
df = pd.DataFrame({
    'customer_id': [101, 102, 103, 104, 105, 106, 107, 108],
    'customer_name': ['Alice Smith', 'Bob Johnson', None, 'Diana Prince', 'Eve Davis', None, 'Grace Wilson', 'Henry Ford'],
    'age': [25, 30, np.nan, 28, np.nan, 35, 42, np.nan],
    'order_date': ['2023-01-15', '2023-02-22', '2023-03-10', None, '2023-05-18', '2023-06-05', '2023-07-12', '2023-08-01'],
    'product': ['Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Laptop', 'Tablet', 'Phone', 'Charger'],
    'quantity': [1, 2, 1, 1, 3, 1, 2, 4],
    'price': ['$999.99', '$49.99', '$79.50', '$299.00', '$1,199.99', '$399.00', '$699.00', '$29.99'],
    'discount': [0.1, 0.05, np.nan, 0.15, 0.2, np.nan, 0.1, 0.0],
    'region': ['North', 'North', 'South', 'East', 'West', 'South', 'North', 'East'],
    'loyalty_member': [True, False, True, True, False, False, True, np.nan]
})
```

Example 1 - Creating a Messy Dataset (Cont.)

```
print(df)
#   customer_id  customer_name  age  order_date  product  quantity  price  discount  region  loyalty_member
# 0          101    Alice Smith  25.0  2023-01-15   Laptop         1   $999.99    0.1  North          True
# 1          102    Bob Johnson  30.0  2023-02-22    Mouse         2   $49.99    0.05  North          False
# 2          103         None   NaN  2023-03-10  Keyboard         1   $79.50    NaN  South          True
# 3          104   Diana Prince  28.0        None  Monitor         1  $299.00    0.15  East          True
# 4          105     Eve Davis   NaN  2023-05-18   Laptop         3  $1,199.99    0.2  West          False
# 5          106         None  35.0  2023-06-05   Tablet         1   $399.00    NaN  South          False
# 6          107   Grace Wilson  42.0  2023-07-12    Phone         2   $699.00    0.1  North          True
# 7          108    Henry Ford   NaN  2023-08-01   Charger         4   $29.99    0.0  East          NaN
```

```
print(df.dtypes)
# customer_id      int64
# customer_name    object
# age              float64
# order_date       object
# product          object
# quantity         int64
# price            object
# discount         float64
# region           object
# loyalty_member    object # Should be bool
```

Integer columns are **not floats by default**; Pandas will keep them as integers (int64) as long as every value in the column is a whole number. **However, they become floats the moment a single NaN (missing value) is introduced.**

```

print("Missing values per column:")
print(df.isna().sum())
# customer_id      0
# customer_name    2
# age              3
# order_date       1
# product          0
# quantity         0
# price            0
# discount         2
# region           0
# loyalty_member   1
# dtype: int64

print("\nRows with ANY missing values:")
print(df.isna().any(axis=1).sum())
# Rows with ANY missing values: 7

print("\nRows where age is missing:")
print(df[df['age'].isna()])
#   customer_id customer_name  age order_date product  quantity  price  discount region loyalty
#_member
# 2           103          None  NaN  2023-03-10  Keyboard         1   $79.50        NaN   South
True
# 4           105      Eve Davis  NaN  2023-05-18   Laptop         3  $1,199.99      0.2    West
False
# 7           108   Henry Ford  NaN  2023-08-01   Charger         4   $29.99      0.0    East
NaN

```

When you put **None** into a **numeric** or **boolean** Series, Pandas automatically converts it to **np.nan** to maintain high performance.

Comparison of Missing Values vs. Literal Strings

Data Entry	Is it "Missing"? (isna())	Data Type
None	Yes	NoneType
np.nan	Yes	float
"None"	No	string (object)
"NaN"	No	string (object)

Example 2 - Deletion

```
df = pd.DataFrame({
    'id': [1, 2, 3, 4, 5],
    'name': ['Alice', 'Bob', None, 'Diana', 'Eve'],
    'score': [85.5, 92.0, np.nan, 88.0, 0.0], # 0.0 is NOT NaN
    'passed': [True, True, False, True, False] # False is NOT NaN
})

print("Original DataFrame:")
print(df)
#   id  name  score  passed
# 0   1  Alice   85.5    True
# 1   2   Bob   92.0    True
# 2   3  None    NaN    False
# 3   4  Diana   88.0    True
# 4   5   Eve    0.0    False

print(f"\nOriginal shape: {df.shape}")
# Original shape: (5, 4)
```

```
print("\nRows with NaN values:")
nan_rows = df[df.isna().any(axis=1)]
print(nan_rows)
#   id  name  score  passed
# 2   3  None    NaN    False # Only row 2!

print(f"\nNumber of rows with NaN: {len(nan_rows)}")
# Number of rows with NaN: 1

# Drop rows with ANY NaN
df_cleaned = df.dropna(how='any')
print(f"\nAfter dropna(how='any'): shape = {df_cleaned.shape}")
# After dropna(how='any'): shape = (4, 4)

print("\nCleaned DataFrame (rows WITHOUT NaN):")
print(df_cleaned)
#   id  name  score  passed
# 0   1  Alice   85.5    True
# 1   2   Bob   92.0    True
# 3   4  Diana   88.0    True
# 4   5   Eve    0.0    False # 0.0 and False are NOT NaN!
```


Example 3 – Imputation

```
df = pd.DataFrame({
    'customer_id': [101, 102, 103, 104, 105, 106, 107, 108],
    'name': ['Alice', 'Bob', None, 'Diana', 'Eve', None, 'Grace', 'Henry'],
    'age': [25, 30, np.nan, 28, np.nan, 35, 42, np.nan],
    'salary': [50000, 60000, 75000, 55000, 48000, 62000, np.nan, 53000],
    'department': ['HR', 'IT', 'IT', 'Finance', 'HR', 'IT', 'Marketing', None],
    'hire_date': ['2020-01-15', '2019-03-22', '2021-06-10', None, '2022-02-28', '2020-11-05', '2021-07-12', '2023-01-10'],
    'performance_score': [85.5, 92.0, 78.5, np.nan, 88.0, np.nan, 95.5, 82.0],
    'active': [True, True, False, True, None, False, True, True]
})

print("Original messy DataFrame:")
print(df)
```

#	customer_id	name	age	salary	department	hire_date	performance_score	active
# 0	101	Alice	25.0	50000.0	HR	2020-01-15	85.5	True
# 1	102	Bob	30.0	60000.0	IT	2019-03-22	92.0	True
# 2	103	None	NaN	75000.0	IT	2021-06-10	78.5	False
# 3	104	Diana	28.0	55000.0	Finance	None	NaN	True
# 4	105	Eve	NaN	48000.0	HR	2022-02-28	88.0	None
# 5	106	None	35.0	62000.0	IT	2020-11-05	NaN	False
# 6	107	Grace	42.0	NaN	Marketing	2021-07-12	95.5	True
# 7	108	Henry	NaN	53000.0	None	2023-01-10	82.0	True

Example 3 – Imputation (Basic Method)

```
# Fill with constant values
df_const = df.copy()
df_const['name'].fillna('Unknown', inplace=True)
df_const['department'].fillna('Unknown', inplace=True)
df_const['active'].fillna(False, inplace=True)

print("After constant imputation (name, department, active):")
print(df_const[['name', 'department', 'active']].head(8))
```

#	name	department	active
# 0	Alice	HR	True
# 1	Bob	IT	True
# 2	Unknown	IT	False
# 3	Diana	Finance	True
# 4	Eve	HR	False
# 5	Unknown	IT	False
# 6	Grace	Marketing	True
# 7	Henry	Unknown	True

❑ If **inplace = True**

Instead of creating and returning a new copy.

❑ Without **inplace = True**

It returns a new Series, but df_const stays unchanged.

```
df_const['name'] = df_const['name'].fillna('Unknown')
```

Example 3 – Imputation (Statistical Method)

```
df_stats = df.copy()

# Mean for normally distributed data
df_stats['salary'].fillna(df_stats['salary'].mean(), inplace=True)

# Median for skewed data (robust to outliers)
df_stats['age'].fillna(df_stats['age'].median(), inplace=True)

# Mode for categorical
dept_mode = df_stats['department'].mode()[0]
df_stats['department'].fillna(dept_mode, inplace=True)

print(f"Mean salary: ${df_stats['salary'].mean():.2f}")
# Mean salary: $57750.00
print(f"Median age: {df_stats['age'].median():.1f}")
# Median age: 30.0
print(f"Mode department: {dept_mode}")
# Mode department: IT
```

```
print("\nAfter statistical imputation:")
print(df_stats[['age', 'salary', 'department']].head(8))

#   age  salary department
# 0  25.0  50000.0         HR
# 1  30.0  60000.0         IT
# 2  30.0  75000.0         IT
# 3  28.0  55000.0    Finance
# 4  30.0  48000.0         HR
# 5  35.0  62000.0         IT
# 6  42.0  57750.0    Marketing # Salary was NaN, filled with mean
# 7  30.0  53000.0         IT  # Department was NaN, filled with mode
```

- **.mode()** is used to find the **most common category** to fill in missing information.
- **.mode()** returns a **Series** containing all values that share the **highest frequency**. If there is a **"tie"** between two departments, the **[0]** simply grabs the one that appears by its original position in the data.

Example 4 – Renaming (Dictionary mapping)

```
df_rename = df.copy()

### Method 1: Rename specific columns
df_renamed = df_rename.rename(columns={
    'customer_id': 'client_id',
    'name': 'full_name',
    'age': 'customer_age',
    'salary': 'annual_salary'
})

print("Method 1: Dictionary mapping:")
print(df_renamed.columns.tolist())
# ['client_id', 'full_name', 'customer_age', 'annual_salary', 'department', 'hire_date', 'performance_score', 'active']
```

Example 4 – Renaming (Bulk renaming with functions)

```
# Convert to snake_case
df_snake = df_rename.copy()
df_snake.columns = [col.lower().replace(' ', '_') for col in df_snake.columns]
print("\nMethod 2: Snake case (all lowercase with underscores):")
print(df_snake.columns.tolist())
# ['customer_id', 'name', 'age', 'salary', 'department', 'hire_date', 'performance_score', 'active']

# Convert to camelCase
df_camel = df_rename.copy()
def to_camel_case(s):
    parts = str(s).replace('_', ' ').split()
    return parts[0].lower() + ''.join(word.title() for word in parts[1:])

df_camel.columns = [to_camel_case(col) for col in df_camel.columns]
print("\nMethod 3: Camel case:")
print(df_camel.columns.tolist())
# ['customerId', 'name', 'age', 'salary', 'department', 'hireDate', 'performanceScore', 'active']

# Add prefix/suffix
df_prefix = df_rename.copy()
df_prefix.columns = 'cust_' + df_prefix.columns
print("\nMethod 4: Add prefix:")
print(df_prefix.columns.tolist())
# ['cust_customer_id', 'cust_name', 'cust_age', 'cust_salary', 'cust_department', 'cust_hire_date',
  'cust_performance_score', 'cust_active']
```

Example 4 – Renaming (In-place renaming)

```
# In-place rename
df_rename.rename(columns={'salary': 'income', 'active': 'is_active'}, inplace=True)
print("\nMethod 5: In-place renaming:")
print(df_rename.columns.tolist())
# ['customer_id', 'name', 'age', 'income', 'department', 'hire_date', 'performance_score', 'is_active']
```

Example 5 – Comprehensive Type Casting

```
# Create a dataset with mixed types
df_types = pd.DataFrame({
    'id': ['001', '002', '003', '004'],
    'price_str': ['$99.99', '149.50', '75.00', '$200.00'],
    'quantity_str': ['5', '10', '3', '8'],
    'date_str': ['2023-01-15', '2023-02-20', '2023-03-25', '2023-04-30'],
    'timestamp_str': ['2023-01-15 10:30:00', '2023-02-20 14:45:00',
                     '2023-03-25 09:15:00', '2023-04-30 16:20:00'],
    'boolean_str': ['True', 'False', 'true', 'false'],
    'percentage_str': ['25%', '50.5%', '75.25%', '100%'],
    'category_str': ['A', 'B', 'A', 'C'],
    'large_int_str': ['1000000', '2000000', '3000000', '4000000']
})

print("Original DataFrame with string types:")
print(df_types.dtypes)

# id            object
# price_str     object
# quantity_str  object
# date_str      object
# timestamp_str object
# boolean_str   object
# percentage_str object
# category_str  object
# large_int_str object
# dtype: object
```

Example 5 – Type Casting - Basic .astype()

```
# Convert string to integer
df_types['id'] = df_types['id'].astype(int)
df_types['quantity_str'] = df_types['quantity_str'].astype(int)

# Convert to float
df_types['price_str'] = df_types['price_str'].str.replace('$', '').astype(float)

# Convert to category (memory efficient)
df_types['category_str'] = df_types['category_str'].astype('category')

print("\nAfter basic type casting:")
print(df_types[['id', 'price_str', 'quantity_str', 'category_str']].dtypes)
# id                int32
# price_str         float64
# quantity_str      int32
# category_str      category
# dtype: object
```


Example 5 – Type Casting - Date/Time Conversion

```
# Convert to datetime
df_types['date'] = pd.to_datetime(df_types['date_str'])
df_types['timestamp'] = pd.to_datetime(df_types['timestamp_str'])

# Extract components
df_types['year'] = df_types['date'].dt.year
df_types['month'] = df_types['date'].dt.month
df_types['day'] = df_types['date'].dt.day
df_types['hour'] = df_types['timestamp'].dt.hour

print("\nDate/time conversion:")
print(df_types[['date_str', 'date', 'year', 'month', 'day']].head())
```

#	date_str	date	year	month	day
# 0	2023-01-15	2023-01-15	2023	1	15
# 1	2023-02-20	2023-02-20	2023	2	20
# 2	2023-03-25	2023-03-25	2023	3	25
# 3	2023-04-30	2023-04-30	2023	4	30

Example 5 – Type Casting - Complex String Parsing

```
# Parse percentage
df_types['percentage'] = df_types['percentage_str'].str.replace('%', '').astype(float) / 100

# Parse currency with thousands separator
df_types['large_int'] = df_types['large_int_str'].astype(int)

# Handle mixed formats
df_types['price_clean'] = pd.to_numeric(
    df_types['price_str'].replace({'\$: ': '', ',': ''}, regex=True),
    errors='coerce'
)

print("\nComplex string parsing:")
print(df_types[['percentage_str', 'percentage', 'large_int_str', 'large_int']])
```

#	percentage_str	percentage	large_int_str	large_int
# 0	25%	0.25	1000000	1000000
# 1	50.5%	0.505	2000000	2000000
# 2	75.25%	0.7525	3000000	3000000
# 3	100%	1.00	4000000	4000000

- ❑ **errors='coerce'** is a safe-conversion switch used in pandas when converting data types.
- ❑ If a value cannot be converted to the target type, replace it with **NaN** instead of raising an error.

Quick Reference Cheat Sheet

```
# IMPUTATION CHEAT SHEET
# df.fillna(0)                                # Fill with constant
# df.fillna(df.mean())                        # Fill with column mean
# df.fillna(df.median())                     # Fill with column median
# df.fillna(method='ffill')                   # Forward fill
# df.fillna(method='bfill')                   # Backward fill
# df.interpolate()                           # Linear interpolation
# SimpleImputer(strategy='mean')              # scikit-learn imputer

# RENAMING CHEAT SHEET
# df.rename(columns={'old':'new'})             # Rename specific
# df.columns = [col.lower() for col in df.columns] # Bulk rename
# df.add_prefix('prefix_')                    # Add prefix
# df.add_suffix('_suffix')                    # Add suffix
# df.columns.str.replace(' ', '_')            # Replace characters

# TYPE CASTING CHEAT SHEET
# df['col'].astype(int)                        # Convert to integer
# df['col'].astype(float)                     # Convert to float
# pd.to_datetime(df['col'])                   # Convert to datetime
# df['col'].astype('category')                 # Convert to category
# pd.to_numeric(df['col'], errors='coerce')    # Safe numeric conversion
# df['col'].astype(str)                       # Convert to string
# df['col'].map({'old':'new'})                  # Custom mapping
```

Forward Fill (ffill) vs Backward Fill (bfill)

❑ Forward Fill (ffill)

Forward fill takes the **last known valid value** and propagates it forward to the next missing cells.

- **Logic:** "Use the value from the past to fill the future."
- **Best for:** Time-series data where the most recent reading is the best guess for the next one (e.g., a sensor reading that hasn't changed).

❑ Backward Fill (bfill)

Backward fill takes the **next known valid value** and propagates it backward to the previous missing cells.

- **Logic:** "Use the value from the future to fill the past."
- **Best for:** Scenarios where you know what the end state was and want to apply it to the lead-up period.

Forward Fill (ffill) vs Backward Fill (bfill)

Method	Logic	Result
Original	Missing data in index 1	[10, NaN, 20]
ffill	Grabs 10 and moves it forward	[10, 10, 20]
bfill	Grabs 20 and moves it backward	[10, 20, 20]

String Operations (.str Accessor)

❑ Vectorized String Processing

pandas provides vectorized string operations, meaning string functions are applied to entire Series at once, not element by element.

```
df['Name'].str.upper()
```

❑ Why this matters:

- Much faster than Python for loops
- Cleaner and more readable code
- Scales well for large datasets

Key String Methods

❑ **.str.contains(pattern)**

Performs **regex-based pattern matching**.

```
df['email'].str.contains('@gmail.com')
```

- Returns a Boolean Series
- Commonly used for filtering rows
- Supports regular expressions

Key String Methods (Cont.)

❑ **.str.split(expand=True)**

Splits strings into multiple columns.

```
df['full_name'].str.split(' ', expand=True)
```

- **expand=True** → returns a DataFrame
- Useful for parsing structured text (names, codes, paths)

Key String Methods (Cont.)

❑ **.str.replace()**

Used to clean dirty text data.

```
df['price_str'].str.replace('$', '', regex=False)
```

Typical use cases:

- Removing currency symbols
- Eliminating unwanted characters
- Normalizing text before type conversion

Complete Examples with Real Data

```
df = pd.DataFrame({
    'employee_id': [101, 102, 103, 104, 105, 106, 107, 108],
    'full_name': ['john doe', 'JANE SMITH', 'Robert Johnson Jr.', 'Maria Garcia-Lopez',
                  'Dr. David Williams', 'Sarah Chen PhD', 'Mike O'Brien', 'Anna van der Berg'],
    'email': ['john.doe@company.com', 'jane.smith@company.com', 'robert.j@company.com',
              'maria.garcia@company.com', 'd.williams@company.com', 'sarah.chen@company.com',
              'mike.obrien@company.com', 'anna.vdb@company.com'],
    'phone': ['(123) 456-7890', '456-789-0123', '789.012.3456', '012-345-6789',
              '+1-234-567-8901', '345.678.9012', 'Invalid Phone', '567-890-1234'],
    'address': ['123 Main St, NY 10001', '456 Oak Ave, LA 90001', '789 Pine Rd, Chicago 60601',
                '101 Maple Dr, Houston 77001', '202 Cedar Ln, Phoenix 85001',
                '303 Birch Blvd, Philadelphia 19101', '404 Spruce Way, San Antonio 78201',
                '505 Redwood Ct, San Diego 92101'],
    'job_title': ['Software Engineer', 'Data Scientist', 'Product Manager',
                  'UX Designer', 'DevOps Engineer', 'Data Analyst',
                  'Frontend Developer', 'Backend Developer'],
    'salary_str': ['$85,000', '$120,000', '$95,500', '$88,000',
                   '$110,000', '$75,000', '$92,000', '$105,000'],
    'notes': ['Joined 2020-01-15, Good performer', 'Team lead, Python expert',
              'Manages 5 people, Excellent communication', 'UI/UX specialist, Creative',
              'AWS certified, Kubernetes', 'SQL expert, Fast learner',
              'React specialist, Good team player', 'Node.js, Microservices']
})
```

```
print(df)
```

#	employee_id	full_name	email	phone	address	job_title	salary_str	notes
# 0	101	john doe	john.doe@company.com	(123) 456-7890	123 Main St, NY 10001	Software Engineer	\$85,000	Joined 2020-01-15, Good performer
# 1	102	JANE SMITH	jane.smith@company.com	456-789-0123	456 Oak Ave, LA 90001	Data Scientist	\$120,000	Team lead, Python expert
# 2	103	Robert Johnson Jr.	robert.j@company.com	789.012.3456	789 Pine Rd, Chicago 60601	Product Manager	\$95,500	Manages 5 people, Excellent communication
# 3	104	Maria Garcia-Lopez	maria.garcia@company.com	012-345-6789	101 Maple Dr, Houston 77001	UX Designer	\$88,000	UI/UX specialist, Creative
# 4	105	Dr. David Williams	d.williams@company.com	+1-234-567-8901	202 Cedar Ln, Phoenix 85001	DevOps Engineer	\$110,000	AWS certified, Kubernetes
# 5	106	Sarah Chen PhD	sarah.chen@company.com	345.678.9012	303 Birch Blvd, Philadelphia 19101	Data Analyst	\$75,000	SQL expert, Fast learner
# 6	107	Mike O'Brien	mike.obrien@company.com	Invalid Phone	404 Spruce Way, San Antonio 78201	Frontend Developer	\$92,000	React specialist, Good team player
# 7	108	Anna van der Berg	anna.vdb@company.com	567-890-1234	505 Redwood Ct, San Diego 92101	Backend Developer	\$105,000	Node.js, Microservices

Example 1 - Basic String Operations

```
df['name_upper'] = df['full_name'].str.upper()
df['name_lower'] = df['full_name'].str.lower()
df['name_title'] = df['full_name'].str.title()

print("\na) Case conversion examples:")
print(df[['full_name', 'name_upper', 'name_lower', 'name_title']].head(3))
#           full_name      name_upper      name_lower      name_title
# 0      john doe      JOHN DOE      john doe      John Doe
# 1      JANE SMITH      JANE SMITH      jane smith      Jane Smith
# 2  Robert Johnson Jr.  ROBERT JOHNSON JR.  robert johnson jr.  Robert Johnson Jr.

# String length
df['name_length'] = df['full_name'].str.len()

# Count occurrences
df['vowel_count'] = df['full_name'].str.lower().str.count(r'[aeiou]')

print("\nb) String metrics:")
print(df[['full_name', 'name_length', 'vowel_count']])
#           full_name      name_length      vowel_count
# 0      john doe      8      3
# 1      JANE SMITH      10      3
# 2  Robert Johnson Jr.      18      5
# 3      Maria Garcia-Lopez      17      7
# 4      Dr. David Williams      18      5
# 5      Sarah Chen PhD      16      4
# 6      Mike O'Brien      13      5
# 7      Anna van der Berg      18      5

# Strip whitespace (also lstrip, rstrip)
df['full_name_clean'] = df['full_name'].str.strip()
```

- **r** means “raw string” in Python.
- A raw string tells Python: “Don’t treat backslashes (\) as escape characters.”

Example 2 - Pattern Matching with .str.contains()

```
# Simple text search
senior_mask = df['job_title'].str.contains('Senior', case=False, na=False)
print("\na) Employees with 'Senior' in title (case-insensitive):")
print(df[senior_mask][['full_name', 'job_title']])
# Empty DataFrame (no one has 'Senior' in title)

# Employees with 'Engineer' in title
engineer_mask = df['job_title'].str.contains('Engineer')
print("\nb) All Engineers:")
print(df[engineer_mask][['full_name', 'job_title']])
#      full_name      job_title
# 0    john doe  Software Engineer
# 4  Dr. David Williams  DevOps Engineer

# Regex patterns
# Find names with PhD or Dr.
academic_mask = df['full_name'].str.contains(r'Dr\.|PhD', regex=True)
print("\nc) Academic titles (Dr. or PhD):")
print(df[academic_mask][['full_name']])
#      full_name
# 4  Dr. David Williams
# 5    Sarah Chen PhD
```

```
# Find emails from specific domain
company_email_mask = df['email'].str.contains(r'@company\.com$', regex=True)
print("\nd) Company email addresses:")
print(df[company_email_mask][['full_name', 'email']])
#      full_name      email
# 0    john doe  john.doe@company.com
# 1    JANE SMITH  jane.smith@company.com
# 2  Robert Johnson Jr.  robert.j@company.com
# 3  Maria Garcia-Lopez  maria.garcia@company.com
# 4  Dr. David Williams  d.williams@company.com
# 5    Sarah Chen PhD  sarah.chen@company.com
# 6    Mike O'Brien  mike.obrien@company.com
# 7  Anna van der Berg  anna.vdb@company.com
```

Example 2 - String Splitting (.str.split())

```
name_split = df['full_name'].str.split(expand=True)
name_split.columns = ['first_name', 'middle', 'last_name', 'suffix']
print(name_split)

#   first_name middle last_name suffix
# 0      john    doe      None     None
# 1      JANE  SMITH      None     None
# 2    Robert Johnson      Jr.     None
# 3      Maria  Garcia  Lopez     None
# 4        Dr. David Williams     None
# 5      Sarah   Chen      PhD     None
# 6      Mike O'Brien      None     None
# 7      Anna    van der Berg

# Better approach with maxsplit
print("\n Split with maxsplit=1 (first name, rest):")
name_split_better = df['full_name'].str.split(n=1, expand=True)
name_split_better.columns = ['first_name', 'rest_of_name']
print(name_split_better)

#   first_name      rest_of_name
# 0      john            doe
# 1      JANE            SMITH
# 2    Robert    Johnson Jr.
# 3      Maria    Garcia-Lopez
# 4        Dr. David Williams
# 5      Sarah      Chen PhD
# 6      Mike      O'Brien
# 7      Anna    van der Berg
```

Split at most 1 time

pandas will split each full_name string into two parts **maximum**:

- **Part 1:** everything before the first separator (**space** by default)
- **Part 2:** everything after that (the rest of the string)

Example 2 - String Splitting (.str.split()) Cont.

```
address_split = df['address'].str.split(',', expand=True)
address_split.columns = ['street', 'city_state_zip']

# Further split city_state_zip
city_split = address_split['city_state_zip'].str.split(' ', n=1, expand=True)
city_split.columns = ['city', 'state_zip']

# Split state_zip
state_zip_split = city_split['state_zip'].str.split(' ', n=1, expand=True)
state_zip_split.columns = ['state', 'zip_code']

# Combine everything
address_df = pd.concat([
    address_split['street'],
    city_split['city'],
    state_zip_split['state'],
    state_zip_split['zip_code']
], axis=1)

print(address_df)
```

#	street	city	state	zip_code
# 0	123 Main St	NY	NY	10001
# 1	456 Oak Ave	LA	LA	90001
# 2	789 Pine Rd,	Chicago	IL	60601
# 3	101 Maple Dr,	Houston	TX	77001
# 4	202 Cedar Ln,	Phoenix	AZ	85001
# 5	303 Birch Blvd	Philadelphia	PA	19101
# 6	404 Spruce Way	San Antonio	TX	78201
# 7	505 Redwood Ct	San Diego	CA	92101

Example 2 - String Replacement (.str.replace())

```
# Clean phone numbers
print("\na) Clean phone numbers (remove non-digits):")
df['phone_clean'] = df['phone'].str.replace(r'^\d', '', regex=True) # Keep only digits
print(df[['phone', 'phone_clean']])
```

#	phone	phone_clean
# 0	(123) 456-7890	1234567890
# 1	456-789-0123	4567890123
# 2	789.012.3456	7890123456
# 3	012-345-6789	0123456789
# 4	+1-234-567-8901	12345678901
# 5	345.678.9012	3456789012
# 6	Invalid Phone	
# 7	567-890-1234	5678901234

The regular expression `r'^\d'` is a powerful "negative" filter. Its goal in this code is to **remove everything that is not a number** (like parentheses, dashes, or spaces) from a phone number string.

Example 2 - String Replacement (.str.replace()) Cont.

```
# Format phone numbers
print("\nb) Format cleaned phone numbers:")
def format_phone(phone):
    if len(phone) == 10:
        return f"({phone[:3]}) {phone[3:6]}-{phone[6:]}"
    elif len(phone) == 11 and phone.startswith('1'):
        return f"+1 ({phone[1:4]}) {phone[4:7]}-{phone[7:]}"
    return phone

df['phone_formatted'] = df['phone_clean'].apply(format_phone)
print(df[['phone_clean', 'phone_formatted']])
```

	phone_clean	phone_formatted
# 0	1234567890	(123) 456-7890
# 1	4567890123	(456) 789-0123
# 2	7890123456	(789) 012-3456
# 3	0123456789	(012) 345-6789
# 4	12345678901	+1 (234) 567-8901
# 5	3456789012	(345) 678-9012
# 6		
# 7	5678901234	(567) 890-1234

Example 2 - String Replacement (.str.replace()) Cont.

```
# Clean salary strings
print("\nc) Convert salary strings to numeric:")
df['salary_numeric'] = df['salary_str'].str.replace(r'[\$,]', '', regex=True).astype(float)
print(df[['salary_str', 'salary_numeric']])
```

#	salary_str	salary_numeric
# 0	\$85,000	85000.0
# 1	\$120,000	120000.0
# 2	\$95,500	95500.0
# 3	\$88,000	88000.0
# 4	\$110,000	110000.0
# 5	\$75,000	75000.0
# 6	\$92,000	92000.0
# 7	\$105,000	105000.0

Example 2 - String Replacement (.str.replace()) Cont.

```
# Remove titles from names
print("\nRemove titles (Dr., PhD) from names:")
df['name_no_title'] = df['full_name'].str.replace(r'Dr\.\s*|PhD\s*', '', regex=True).str.strip()
print(df[['full_name', 'name_no_title']].head())
```

#	full_name	name_no_title
# 0	john doe	john doe
# 1	JANE SMITH	JANE SMITH
# 2	Robert Johnson Jr.	Robert Johnson Jr.
# 3	Maria Garcia-Lopez	Maria Garcia-Lopez
# 4	Dr. David Williams	David Williams

The regex pattern: `r'Dr\.\s*|PhD\s*'`

This pattern has two alternatives separated by | (OR):

A) `Dr\.\s*`

- **Dr** matches the letters “Dr”
- **\.** matches a literal dot (.) - The **dot** is special in regex (means “any character”), so we escape it as **\.** to mean a real period.
- **\s*** matches 0 or more whitespace characters (spaces, tabs, etc.)

Quick Reference Cheat Sheet

BASIC OPERATIONS:

```
df['col'].str.upper()      # Convert to uppercase
df['col'].str.lower()      # Convert to lowercase
df['col'].str.title()      # Title case
df['col'].str.len()        # String length
df['col'].str.strip()      # Remove whitespace
```

PATTERN MATCHING:

```
df['col'].str.contains('pattern')  # Check for substring
df['col'].str.startswith('prefix') # Starts with
df['col'].str.endswith('suffix')   # Ends with
df['col'].str.match('regex')        # Full string match
```

SPLITTING:

```
df['col'].str.split(',')          # Split by delimiter
df['col'].str.split(n=2)           # Split with max splits
df['col'].str.split(expand=True)   # Return DataFrame
```

REPLACEMENT:

```
df['col'].str.replace('old', 'new') # Simple replace
df['col'].str.replace(r'\\d', '', regex=True) # Regex replace
```

EXTRACTION:

```
df['col'].str.extract(r'(\d+)')    # Extract pattern
df['col'].str.extractall(r'(\w+)')  # Extract all matches
```

SLICING:

```
df['col'].str[0:5]                  # Get substring
df['col'].str.slice(0, 5)            # Slice method
```

OTHER:

```
df['col'].str.cat(sep=', ')         # Concatenate
df['col'].str.get_dummies()          # One-hot encoding
df['col'].str.find('sub')             # Find position
df['col'].str.count('a')              # Count occurrences
"""
```

The Philosophy of Selection

Explicit vs. Implicit Selection

□ `.loc[row_label, col_label]` — Explicit (Label-based)

Give me the row labeled `Index_5` and the column `Price`.

```
df.loc['Index_5', 'Price']
```

Key properties:

- Uses labels, not positions
- Includes the stop endpoint in slicing
- Preferred for production code (clear, robust, readable)

The Philosophy of Selection (Cont.)

□ `.iloc[row_pos, col_pos]` — Implicit (Position-based)

Give me row 5 and column 2.

```
df.iloc[5, 2]
```

Key properties:

- Uses integer positions (0 to N-1)
- Excludes the stop endpoint (standard Python behavior)
- Fast and predictable for numeric indexing

The Philosophy of Selection (Cont.)

□ Direct Column Access

```
df['col']           # Returns a Series  
df[['col1', 'col2']] # Returns a DataFrame
```

Rule of thumb:

- Single bracket → Series
- Double brackets → DataFrame

Example 1

```
np.random.seed(42)
df = pd.DataFrame({
    'Product': ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry', 'Fig'],
    'Price': [1.20, 0.50, 3.75, 2.30, 4.10, 2.80],
    'Quantity': [100, 250, 75, 150, 40, 120],
    'Category': ['Fruit', 'Fruit', 'Fruit', 'Fruit', 'Berry', 'Fruit']
}, index=['Store_A', 'Store_B', 'Store_C', 'Store_D', 'Store_E', 'Store_F'])

print("=== ORIGINAL DATAFRAME ===")
print(df)
```

#	Product	Price	Quantity	Category
# Store_A	Apple	1.20	100	Fruit
# Store_B	Banana	0.50	250	Fruit
# Store_C	Cherry	3.75	75	Fruit
# Store_D	Date	2.30	150	Fruit
# Store_E	Elderberry	4.10	40	Berry
# Store_F	Fig	2.80	120	Fruit

What np.random.seed(42)

Does

- It sets the starting point for NumPy's random number generator.
- You will **get the same sequence of random numbers** every time you run your code if you set the same seed.
- The seed value could be **any positive** number from **0** to **2^{32}**

Example 1 – Direct Access

```
# ===== 1. DIRECT ACCESS (SINGLE COLUMN) =====  
print("1. DIRECT ACCESS: df['Price']")  
price_series = df['Price']  
print(f"Type: {type(price_series)}") # Type: <class 'pandas.core.series.Series'>  
print(f"First 3 values:\n{price_series.head(3)}")  
# Store_A    1.20  
# Store_B    0.50  
# Store_C    3.75
```

```
# ===== 2. DIRECT ACCESS (MULTIPLE COLUMNS) =====  
print("2. DIRECT ACCESS: df[['Product', 'Price']]")  
two_cols = df[['Product', 'Price']]  
print(f"Type: {type(two_cols)}") # Type: <class 'pandas.core.frame.DataFrame'>  
print(f"Shape: {two_cols.shape}") # Shape: (6, 2)  
print(two_cols)  
#          Product  Price  
# Store_A    Apple    1.20  
# Store_B    Banana    0.50  
# Store_C    Cherry    3.75  
# Store_D     Date    2.30  
# Store_E  Elderberry    4.10  
# Store_F     Fig     2.80
```


Example 1 – .loc[] - Explicit Label Selection

```
# Single cell
print("Single cell: df.loc['Store_C', 'Price']")
value = df.loc['Store_C', 'Price']
print(f"Value: {value}")           # Value: 3.75
print(f"Type: {type(value)}")     # Type: <class 'numpy.float64'>

# Single row
print("\nSingle row: df.loc['Store_B']")
row = df.loc['Store_B']
print(f"Type: {type(row)}")       # Type: <class 'pandas.core.series.Series'>
print(row)
# Product      Banana
# Price         0.50
# Quantity     250
# Category     Fruit

# Multiple rows and specific columns
print("\nMultiple rows, specific columns: df.loc[['Store_A', 'Store_E'], ['Product', 'Quantity']]")
subset = df.loc[['Store_A', 'Store_E'], ['Product', 'Quantity']]
print(subset)
#           Product  Quantity
# Store_A    Apple      100
# Store_E  Elderberry      40
```

Example 1 – .Loc[] - Slicing (Inclusive!)

```
print("Label slice: df.loc['Store_B':'Store_D', 'Price':'Quantity']")
slice_result = df.loc['Store_B':'Store_D', 'Price':'Quantity']
print(slice_result)
#           Price  Quantity
# Store_B    0.50        250
# Store_C    3.75         75
# Store_D    2.30        150

print(f"\nNote: 'Store_D' is INCLUDED (3 stores total)")
print(f"Shape of slice: {slice_result.shape}") # Shape of slice: (3, 2)

print("\n" + "="*50 + "\n")
```

Example 1 – .iloc[] - Implicit Position Selection

```
print("Single cell: df.iloc[2, 1] (row 2, col 1)")
value_iloc = df.iloc[2, 1]
print(f"Value: {value_iloc}")           # Value: 3.75 (same as df.loc['Store_C', 'Price'])

print("\nRow slice: df.iloc[1:4] (rows 1, 2, 3)")
row_slice = df.iloc[1:4]
print(row_slice)
```

#	Product	Price	Quantity	Category
# Store_B	Banana	0.50	250	Fruit
# Store_C	Cherry	3.75	75	Fruit
# Store_D	Date	2.30	150	Fruit

```
print(f"\nNote: Position 4 is EXCLUDED (Python standard)")
print(f"Shape: {row_slice.shape}")      # Shape: (3, 4)
```

Mixed selection

```
print("\nMixed: df.iloc[0:3, [0, 2]] (rows 0-2, cols 0 and 2)")
```

```
mixed = df.iloc[0:3, [0, 2]]
```

```
print(mixed)
```

#	Product	Quantity
# Store_A	Apple	100
# Store_B	Banana	250
# Store_C	Cherry	75

Boolean Filtering (Masking)

□ Core Logic

DataFrame[Boolean Series]

- Rows are selected where the condition evaluates to True.

□ Single Condition

```
df[df['Price'] > 100]
```

Boolean Filtering (Masking)

❏ Compound Conditions

➤ Important rule

- You must use: **Parentheses ()**
- Bitwise operators: **&** (AND), **|** (OR), **~** (NOT)

➤ Correct

```
df[(df['Price'] > 100) & (df['Region'] == 'East')]
```

➤ InCorrect

```
df[df['Price'] > 100 and df['Region'] == 'East']
```

➤ Python's and/or do not work with pandas Series.

Advanced Filtering

❑ `.isin()` — SQL-style IN

```
df[df['Region'].isin(['East', 'West'])]
```

❑ `.query()` — Expression-based syntax

```
df.query('Price > 100 & Region == "East"')
```

Advantages:

- Cleaner syntax
- Easier to read for complex conditions

Example 1 – Original DataFrame

#	Product	Price	Quantity	Region	Category	Stock_Status
# 0	Laptop	1200	15	West	Electronics	High
# 1	Tablet	650	25	East	Electronics	Medium
# 2	Phone	800	40	North	Electronics	Low
# 3	Monitor	300	30	South	Electronics	High
# 4	Keyboard	85	50	West	Accessory	Medium
# 5	Mouse	45	100	East	Accessory	High
# 6	Headphones	150	60	North	Accessory	Low
# 7	Charger	30	200	South	Accessory	High
# 8	Laptop	1250	10	West	Electronics	Medium
# 9	Tablet	700	20	East	Electronics	High
# 10	Phone	850	35	North	Electronics	Low
# 11	Monitor	320	25	South	Electronics	Medium
# 12	Keyboard	90	45	West	Accessory	High

Example 1 – Single Condition Filtering

```
expensive_items = df[df['Price'] > 100]
print(f"Items priced over $100: {len(expensive_items)} out of {len(df)} items")
print(expensive_items[['Product', 'Price', 'Region']])
```

#	Product	Price	Region
# 0	Laptop	1200	West
# 1	Tablet	650	East
# 2	Phone	800	North
# 3	Monitor	300	South
# 6	Headphones	150	North
# 8	Laptop	1250	West
# 9	Tablet	700	East
# 10	Phone	850	North
# 11	Monitor	320	South

```
# Understanding the boolean mask
print("\nBoolean mask: df['Price'] > 100")
mask = df['Price'] > 100
print(mask)
```

# 0	True
# 1	True
# 2	True
# 3	True
# 4	False
# 5	False
# 6	True
# 7	False
# 8	True
# 9	True
# 10	True
# 11	True
# 12	False

Example 1 – COMPOUND LOGIC - CORRECT Vs INCORRECT

```
# CORRECT: Using bitwise operators with parentheses
print("CORRECT: df[(df['Price'] > 100) & (df['Region'] == 'East')]")
correct_filter = df[(df['Price'] > 100) & (df['Region'] == 'East')]
print(correct_filter[['Product', 'Price', 'Region']])

#   Product  Price Region
# 1  Tablet    650   East
# 9  Tablet    700   East

# INCORRECT: This would cause an error - uncomment to see
try:
    print("\nAttempting INCORRECT: df[df['Price'] > 100 and df['Region'] == 'East']")
    # incorrect = df[df['Price'] > 100 and df['Region'] == 'East'] # ValueError!
except ValueError as e:
    print(f"Error (as expected): {str(e)[:50]}...")

print("\nWhy? Python's 'and' tries to evaluate Series as single boolean.")
print("Use & for element-wise AND, | for OR, ~ for NOT")
```

Example 1 – More compound examples

```
print("\nPrice > 100 AND Category is Electronics")
filter1 = df[(df['Price'] > 100) & (df['Category'] == 'Electronics')]
print(f"Results: {len(filter1)} items")
print(filter1[['Product', 'Price', 'Category']])
#   Product  Price  Category
# 0  Laptop   1200  Electronics
# 1  Tablet    650  Electronics
# 2   Phone    800  Electronics
# 3 Monitor    300  Electronics
# 8  Laptop   1250  Electronics
# 9  Tablet    700  Electronics
# 10 Phone    850  Electronics
# 11 Monitor   320  Electronics

print("\nPrice > 100 OR Quantity < 30")
filter2 = df[(df['Price'] > 100) | (df['Quantity'] < 30)]
print(f"Results: {len(filter2)} items")

print("\nNOT in West region: df[~(df['Region'] == 'West')]")
not_west = df[~(df['Region'] == 'West')]
print(f"Items not in West region: {len(not_west)} items")
```

Example 1 – .isin() Method - Sql-like In Clause

```
# Filter for specific products
print("Products that are either Laptop, Phone, or Headphones:")
target_products = ['Laptop', 'Phone', 'Headphones']
product_filter = df[df['Product'].isin(target_products)]
print(product_filter[['Product', 'Price', 'Quantity']])

#      Product  Price  Quantity
# 0      Laptop   1200         15
# 2        Phone    800         40
# 6  Headphones    150         60
# 8      Laptop   1250         10
# 10     Phone    850         35

print("\nRegions in East or West:")
region_filter = df[df['Region'].isin(['East', 'West'])]
print(f"East/West items: {len(region_filter)} out of {len(df)}")
print(region_filter[['Product', 'Region']].head())

#      Product Region
# 0  Laptop    West
# 1  Tablet   East
# 4  Keyboard  West
# 5   Mouse   East
# 8  Laptop    West
```

Example 1 – Using .isin() with negation

```
print("\nProducts NOT in the list (combine with ~):")
not_target = df[~df['Product'].isin(target_products)]
print(f"Other products: {not_target['Product'].unique().tolist()}")
# Other products: ['Tablet', 'Monitor', 'Keyboard', 'Mouse', 'Charger']
```

Example 1 – .Query() Method

```
print("\nComplex: df.query('Price > 100 & Region == \"East\" & Quantity < 30')")
query2 = df.query('Price > 100 & Region == "East" & Quantity < 30')
print(query2[['Product', 'Price', 'Region', 'Quantity']])
#   Product  Price Region  Quantity
# 1  Tablet   650   East      25
# 9  Tablet   700   East      20

# Using variables in query
print("\nUsing @variable in query:")
min_price = 500
max_quantity = 30
query3 = df.query('Price > @min_price & Quantity < @max_quantity')
print(f"Price > ${min_price} AND Quantity < {max_quantity}: {len(query3)} items")
print(query3[['Product', 'Price', 'Quantity']])
#   Product  Price  Quantity
# 1  Tablet   650         25
# 8  Laptop  1250         10
# 9  Tablet   700         20
```

Sorting & Ranking

❑ Sorting: Multi-level sorting

sort_values() sorts a DataFrame by one or more columns.

- Supports multiple columns
- Independent sort directions
- **The following Code:**

```
df.sort_values(  
    by=['Col1', 'Col2'],  
    ascending=[True, False]  
)
```

- Sort the DataFrame first by Col1, then (if there are ties) sort those groups by Col2.
- **True for Col1** → sort Col1 in ascending order (smallest → largest)
- **False for Col2** → sort Col2 in descending order (largest → smallest)

Sorting & Ranking

□ Sorting by index

```
df.sort_index()
```

- Aligns data by row labels
- Useful after joins and reindexing

Sorting & Ranking

❑ Ranking: .rank()

Assigns numerical ranks (1st, 2nd, ...) to values.

```
df['rank'] = df['Return'].rank(method='dense')
```

➤ Common methods:

- **min** → same rank, gaps allowed
- **dense** → same rank, no gaps “If two values tie for rank 1, the next rank is 2 (not 3).”

Example 1 – Original DataFrame

#	Product	Brand	Price	Units_Sold	Profit_Margin	Region
# S10	Laptop	Apple	1299	150	0.35	West
# S05	Tablet	Samsung	799	300	0.25	East
# S01	Phone	Apple	999	500	0.30	North
# S08	Monitor	Dell	349	200	0.20	South
# S03	Keyboard	Logitech	129	450	0.40	West
# S06	Mouse	Logitech	49	800	0.50	East
# S02	Headphones	Sony	299	350	0.25	North
# S09	Charger	Anker	29	1200	0.15	South
# S04	Laptop	Dell	899	180	0.28	West
# S07	Tablet	Microsoft	899	250	0.32	East

Example 1 – Basic Sorting

```
print("A. Sort by Price (ascending - default):")
price_sorted = df.sort_values(by='Price')
print(price_sorted[['Product', 'Brand', 'Price']])
```

#	Product	Brand	Price
# S09	Charger	Anker	29
# S06	Mouse	Logitech	49
# S03	Keyboard	Logitech	129
# S02	Headphones	Sony	299
# S08	Monitor	Dell	349
# S04	Laptop	Dell	899
# S07	Tablet	Microsoft	899
# S05	Tablet	Samsung	799
# S01	Phone	Apple	999
# S10	Laptop	Apple	1299

Note: 799 comes after 899 (ascending)

```
print("\nB. Sort by Price (descending):")
price_desc = df.sort_values(by='Price', ascending=False)
print(price_desc[['Product', 'Brand', 'Price']].head(3))
```

#	Product	Brand	Price
# S10	Laptop	Apple	1299
# S01	Phone	Apple	999
# S04	Laptop	Dell	899

Example 1 – Multi-level Sorting

```
multi_sorted = df.sort_values(by=['Brand', 'Price'], ascending=[True, False])  
print(multi_sorted[['Brand', 'Product', 'Price']])
```

Brand	Product	Price

Apple	Laptop	1299
Apple	Phone	999
Anker	Charger	29
Dell	Laptop	899
Dell	Monitor	349
Logitech	Keyboard	129
Logitech	Mouse	49
Microsoft	Tablet	899
Samsung	Tablet	799
Sony	Headphones	299

Example 1 – Complex Sorting

```
print("Complex: Sort by Region (custom order), then Profit Margin (high to low):")
region_order = {'West': 1, 'East': 2, 'North': 3, 'South': 4}
df['Region_Order'] = df['Region'].map(region_order)
complex_sorted = df.sort_values(by=['Region_Order', 'Profit_Margin'],
                                ascending=[True, False])
print(complex_sorted[['Region', 'Product', 'Profit_Margin']])
```

#	Region	Product	Profit_Margin
# S03	West	Keyboard	0.40
# S10	West	Laptop	0.35
# S04	West	Laptop	0.28
# S06	East	Mouse	0.50
# S07	East	Tablet	0.32
# S05	East	Tablet	0.25
# S01	North	Phone	0.30
# S02	North	Headphones	0.25
# S08	South	Monitor	0.20
# S09	South	Charger	0.15

```
df = df.drop('Region_Order', axis=1) # Clean up
```

Example 1 – Sorting By Index

```
print("A. Sort index (alphabetical/numerical order):")
index_sorted = df.sort_index()
print(index_sorted.index.tolist()) # ['S01', 'S02', 'S03', 'S04', 'S05', 'S06', 'S07', 'S08', 'S09', 'S10']
print(index_sorted[['Product', 'Brand']].head(3))
#      Product Brand
# S01   Phone  Apple
# S02 Headphones  Sony
# S03  Keyboard Logitech

print("\nB. Sort index in descending order:")
index_desc = df.sort_index(ascending=False)
print(index_desc.index.tolist()[:3]) # ['S10', 'S09', 'S08']
print(index_desc[['Product', 'Brand']].head(3))
#      Product Brand
# S10   Laptop  Apple
# S09   Charger  Anker
# S08   Monitor   Dell
```

Example 2 – Ranking (Default)

```
# Create a simpler series for ranking demonstration
sales_data = pd.Series([100, 150, 150, 200, 250, 250, 250, 300],
                       index=['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])
print("Sales Data with ties:")
print(sales_data)

# A    100
# B    150
# C    150
# D    200
# E    250
# F    250
# G    250
# H    300

print("\nA. Default ranking (average method for ties):")
rank_default = sales_data.rank()
print(rank_default)

# A    1.0 # Lowest value = rank 1
# B    2.5 # Tied with C: (2+3)/2 = 2.5
# C    2.5 # Tied with B
# D    4.0
# E    6.0 # Three-way tie: (5+6+7)/3 = 6.0
# F    6.0
# G    6.0
# H    8.0
```

Example 2 – Ranking (min & dense)

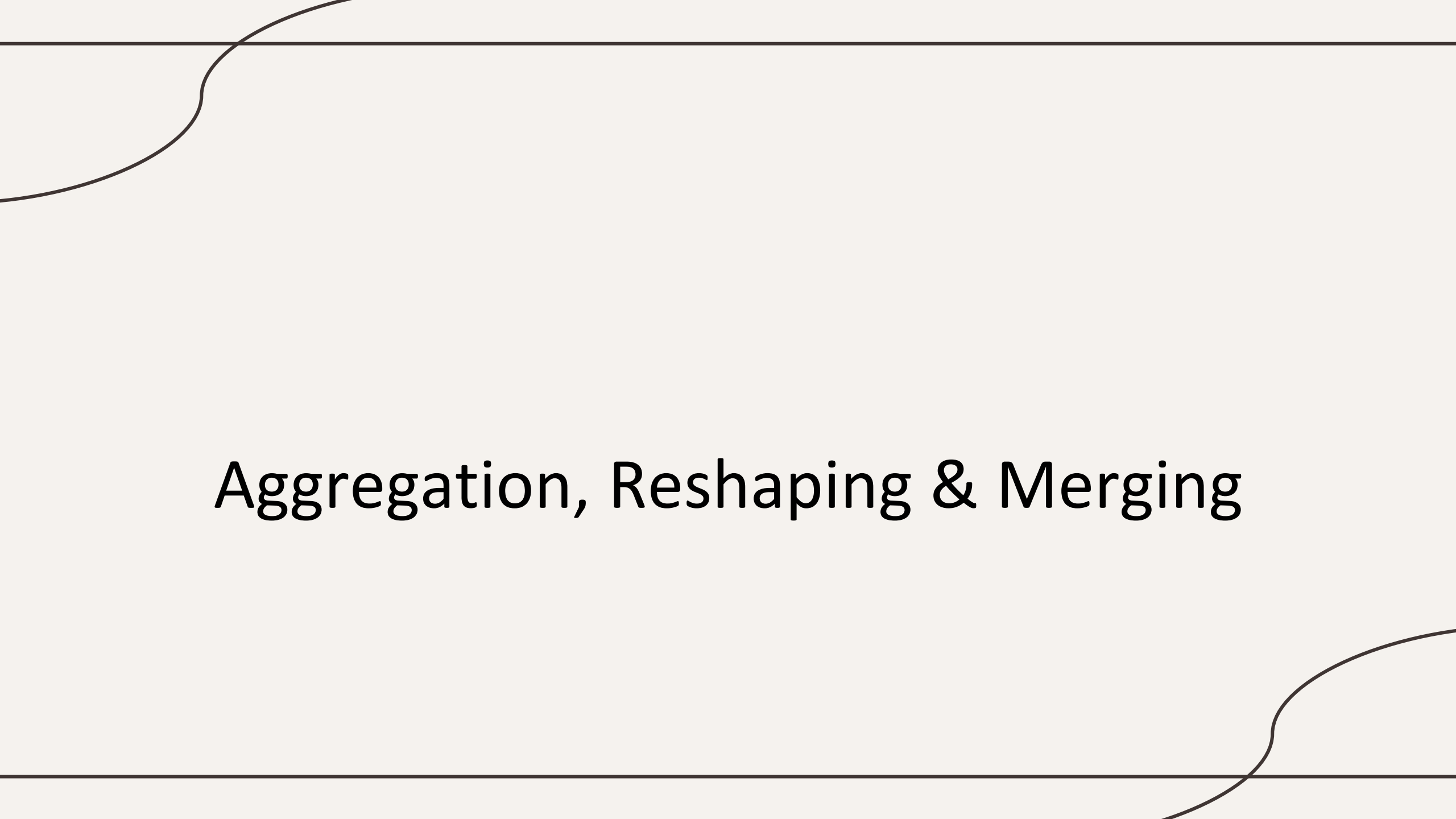
```
print("\nB. 'min' method (lowest rank for ties):")
rank_min = sales_data.rank(method='min')
print(rank_min)
# A    1.0
# B    2.0  # Tied values get the minimum rank (2)
# C    2.0
# D    4.0
# E    5.0  # All three get rank 5
# F    5.0
# G    5.0
# H    8.0

print("\nC. 'dense' method (no gaps in ranking):")
rank_dense = sales_data.rank(method='dense')
print(rank_dense)
# A    1.0
# B    2.0  # Still rank 2
# C    2.0
# D    3.0  # Note: No gap! Jumps to 3 (not 4)
# E    4.0  # Jumps to 4 (not 5 or 6)
# F    4.0
# G    4.0
# H    5.0  # Only 5 unique ranks total
```

Example 2 – Ranking (first & descending)

```
print("\nD. 'first' method (order of appearance breaks ties):")
rank_first = sales_data.rank(method='first')
print(rank_first)
# A    1.0
# B    2.0 # B appears first in tie
# C    3.0 # C appears second
# D    4.0
# E    5.0 # E appears first in 3-way tie
# F    6.0 # F appears second
# G    7.0 # G appears third
# H    8.0

print("\nE. Ranking in descending order (highest value = rank 1):")
rank_desc = sales_data.rank(ascending=False)
print(rank_desc)
# A    8.0 # Lowest value gets highest rank number
# B    6.5 # (6+7)/2 = 6.5
# C    6.5
# D    4.0
# E    2.0 # (2+3+4)/3 = 3.0
# F    2.0
# G    2.0
# H    1.0 # Highest value = rank 1
```


The image features two thin, dark horizontal lines. The top line starts on the left edge and ends with a smooth, upward-curving arc. The bottom line starts with a smooth, downward-curving arc on the left and ends on the right edge. These lines frame the central text.

Aggregation, Reshaping & Merging

The Split-Apply-Combine Strategy (GroupBy)

❑ Core Concept

Split – Apply – Combine is a fundamental data-analysis paradigm.

➤ Split

Divide the dataset into groups based on one or more keys
(e.g., *Department, Region, Category*)

➤ Apply

Apply a function to each group
(*sum, mean, count, max, custom functions*)

```
df.groupby('Key_Column').agg(func)
```

➤ Combine

Combine the results into a single DataFrame or Series

- **groupby('Key_Column')** → Splits the DataFrame into groups based on **unique values in Key_Column**.

Aggregation with .agg()

Multiple functions on one column

```
df.groupby('Department')['Salary'].agg(['mean', 'sum'])
```

Different functions on different columns

```
df.groupby('Department').agg({  
    'Price': 'mean',  
    'Qty': 'sum'  
})
```

❑ Why .agg() is powerful:

- **Flexible**
- **Explicit**
- **Scales well to complex analytics**

Example – Original DataSet

#	Region	Department	Salesperson	Sales_Amount	Quantity	Month
# 0	North	Electronics	Alice	1500	15	Jan
# 1	North	Furniture	Bob	2200	22	Jan
# 2	North	Electronics	Charlie	1800	18	Feb
# 3	South	Furniture	David	1950	19	Feb
# 4	South	Clothing	Eve	800	40	Mar
# 5	South	Clothing	Frank	1200	60	Mar
# 6	East	Electronics	Grace	2100	21	Jan
# 7	East	Clothing	Henry	950	95	Feb
# 8	West	Electronics	Ivy	1700	17	Mar
# 9	West	Furniture	Jack	2400	24	Jan
# 10	West	Clothing	Kate	1100	55	Feb
# 11	West	Electronics	Leo	1600	16	Mar

Example – Groupby Object

```
print("A. Create GroupBy object by Region:")
grouped_by_region = df.groupby('Region')
print(f"Type: {type(grouped_by_region)}") # <class 'pandas.core.groupby.generic.DataFrameGroupBy'>
print(f"Number of groups: {grouped_by_region.ngroups}") # 4
print(f"Group names: {list(grouped_by_region.groups.keys())}") # ['East', 'North', 'South', 'West']

print("\nB. See what's in the 'North' group:")
print("Grouped object is LAZY - doesn't compute until we ask:")
north_group = grouped_by_region.get_group('North')
print(north_group)
#   Region  Department Salesperson  Sales_Amount  Quantity Month
# 0  North  Electronics      Alice          1500         15   Jan
# 1  North   Furniture       Bob           2200         22   Jan
# 2  North  Electronics      Charlie         1800         18   Feb

print("\nC. Iterate through groups (for debugging):")
for name, group in df.groupby('Region'):
    print(f"\nRegion: {name} ({len(group)} records)")
    print(group[['Department', 'Sales_Amount']].head(2))
```

Example – Basic Aggregation - Single Function")

```
print("A. Total sales by Region:")
total_by_region = df.groupby('Region')['Sales_Amount'].sum()
print(total_by_region)
# Region
# East      3050
# North     5500
# South     3950
# West      6800
# Name: Sales_Amount, dtype: int64

print(f"\nType: {type(total_by_region)}") # Series (single column result)

print("\nB. Average sales by Department:")
avg_by_dept = df.groupby('Department')['Sales_Amount'].mean()
print(avg_by_dept)
# Department
# Clothing      1012.500000
# Electronics   1750.000000
# Furniture     2183.333333
# Name: Sales_Amount, dtype: float64
```

Example – Basic Aggregation - Multiple statistic")

```
print("\nC. Multiple statistics for Quantity by Region:")
qty_stats = df.groupby('Region')['Quantity'].agg(['sum', 'mean', 'min', 'max', 'count'])
print(qty_stats)
```

#		sum	mean	min	max	count
#	Region					
#	East	116	58.000000	21	95	2
#	North	55	18.333333	15	22	3
#	South	119	59.500000	19	60	3
#	West	112	28.000000	16	55	4

```
print("A. Same function for multiple columns:")
region_stats = df.groupby('Region')[['Sales_Amount', 'Quantity']].agg(['sum', 'mean'])
print(region_stats)
```

#		Sales_Amount		Quantity	
#		sum	mean	sum	mean
#	Region				
#	East	3050	1525.000000	116	58.000000
#	North	5500	1833.333333	55	18.333333
#	South	3950	1316.666667	119	59.500000
#	West	6800	1700.000000	112	28.000000

Example – Aggregation - Different functions")

```
custom_agg = df.groupby('Region').agg({
    'Sales_Amount': ['sum', 'mean'],      # Two functions for Sales_Amount
    'Quantity': 'sum',                    # Single function for Quantity
    'Salesperson': 'count'                # Count of salespeople
})
print(custom_agg)
```

	Sales_Amount		Quantity		Salesperson
	sum	mean	sum	count	
Region					
East	3050	1525.000000	116	2	
North	5500	1833.333333	55	3	
South	3950	1316.666667	119	3	
West	6800	1700.000000	112	4	

```
print("\nC. Named aggregations (pandas 0.25+):")
named_agg = df.groupby('Region').agg(
    total_sales=('Sales_Amount', 'sum'),
    avg_sales=('Sales_Amount', 'mean'),
    salespeople=('Salesperson', 'nunique'), # Count unique salespeople
    avg_quantity=('Quantity', 'mean')
)
print(named_agg)
```

	total_sales	avg_sales	salespeople	avg_quantity
Region				
East	3050	1525.000000	2	58.000000
North	5500	1833.333333	3	18.333333
South	3950	1316.666667	3	59.500000
West	6800	1700.000000	4	28.000000

Reshaping Data (Pivot & Melt)

Wide Format

- One row per entity (e.g., per customer, per product).
- Multiple columns represent different measurements or time points.
- Spreadsheet style
- **Example:** Jan_Sales, Feb_Sales, Mar_Sales

Long Format

- One row per observation.
- Fields: ID, variable, value
- Database style
- Easier for plotting, modeling, and group operations.

Reshaping Data (Cont.)

❑ **Pivoting (Long → Wide)**

The safest and most flexible method to create a wide format.

- Handles duplicate entries by aggregating them.
- Common arguments:

```
df.pivot_table(  
    index='Row_Identifier',      # Rows  
    columns='Column_Identifier', # New columns  
    values='Data',              # Values to fill table  
    aggfunc='mean'              # How to aggregate duplicates  
)
```

When to use:

- Creating Excel-style tables
- Time series spreading across columns
- Matrix-format reporting (Region × Product × Revenue)

Reshaping Data (Cont.)

❑ Wide Format (Spreadsheet Style)

- Each column is a variable. One row per subject.
- **Example: Monthly Sales (Wide)**

Product	Jan_Sales	Feb_Sales	Mar_Sales
A	100	120	130
B	80	90	95
C	150	160	170

Reshaping Data (Cont.)

❑ Melting (Wide → Long)

Converts columns into rows — “unpivoting.”

- Turns wide-format data into tidy long-format data.
- Common arguments:

```
df.melt(  
    id_vars=['Keep_These_Columns'],      # Identifiers (stay the same)  
    value_vars=['Unpivot_These'],        # Columns to unpivot  
    var_name='Variable',                  # Name for melted column  
    value_name='Value'                    # Name for new value column  
)
```

When to use:

- **Preparing data for:**
 - Groupby
 - statistical models
- Transforming monthly/weekly columns into a single Month column

Reshaping Data (Cont.)

❑ LONG FORMAT (Database Style)

Each row is one observation.

❑ Same dataset converted to Long:

Product	Month	Sales
A	Jan	100
A	Feb	120
A	Mar	130
B	Jan	80
B	Feb	90
B	Mar	95
C	Jan	150
C	Feb	160
C	Mar	170

Example 1: Pivot Table (Long → Wide)

❑ Starting from this wide table:

Product	Q1	Q2	Q3	Q4
A	10	12	15	14
B	9	11	13	16

❑ Result (Long table)

Product	Quarter	Sales
A	Q1	10
A	Q2	12
A	Q3	15
A	Q4	14
B	Q1	9
B	Q2	11
B	Q3	13
B	Q4	16

❑ Melt it:

```
df.melt(  
    id_vars='Product',  
    value_vars=['Q1', 'Q2', 'Q3', 'Q4'],  
    var_name='Quarter',  
    value_name='Sales'  
)
```

Example 2: Melt Example (Wide → Long)

❑ If we start from Long data:

Product	Month	Sales
A	Jan	100
A	Jan	110
A	Feb	120
B	Jan	80

❑ Pivot Table:

```
df.pivot_table(  
    index='Product',  
    columns='Month',  
    values='Sales',  
    aggfunc='mean'  
)
```

❑ Result:

Duplicates averaged in Jan:

Product	Jan	Feb
A	105	120
B	80	NaN

Example 3: Pivot - Original DataSet

```
long_df = pd.DataFrame({
    'Student': ['Alice', 'Alice', 'Alice', 'Bob', 'Bob', 'Bob', 'Charlie', 'Charlie', 'Charlie'],
    'Subject': ['Math', 'Science', 'English', 'Math', 'Science', 'English', 'Math', 'Science', 'English'],
    'Score': [90, 85, 92, 75, 80, 78, 95, 88, 91],
    'Year': [2022, 2022, 2022, 2022, 2022, 2022, 2022, 2022, 2022]
})

print("LONG Format (Database style):")
print("One row per observation:")
print(long_df)
```

#	Student	Subject	Score	Year
# 0	Alice	Math	90	2022
# 1	Alice	Science	85	2022
# 2	Alice	English	92	2022
# 3	Bob	Math	75	2022
# 4	Bob	Science	80	2022
# 5	Bob	English	78	2022
# 6	Charlie	Math	95	2022
# 7	Charlie	Science	88	2022
# 8	Charlie	English	91	2022

Example 3: Pivot - Wide Format

```
# Convert to WIDE format (spreadsheet-style)
wide_df = long_df.pivot_table(
    index='Student',      # Rows
    columns='Subject',    # Columns
    values='Score',       # Values to fill
    aggfunc='first'       # Use first value (no duplicates here)
)

print("WIDE Format (Spreadsheet style):")
print("One row per student, one column per subject:")
print(wide_df)
# Subject  English  Math  Science
# Student
# Alice      92     90     85
# Bob       78     75     80
# Charlie   91     95     88

print(f"\nShape: {wide_df.shape}") # (3, 3) - 3 students, 3 subjects
```

Example 4: Pivot - Original DataSet

```
# Create data with duplicates (real-world scenario)
sales_data = pd.DataFrame({
    'Date': ['2023-01-01', '2023-01-01', '2023-01-01', '2023-01-01',
            '2023-01-02', '2023-01-02', '2023-01-02'],
    'Product': ['Laptop', 'Tablet', 'Laptop', 'Phone',
               'Tablet', 'Laptop', 'Phone'],
    'Region': ['North', 'North', 'South', 'North',
              'South', 'North', 'South'],
    'Revenue': [1000, 500, 1200, 300, 550, 1100, 350],
    'Quantity': [5, 10, 6, 15, 11, 7, 18]
})

print("Sales Data (LONG format with potential duplicates):")
print(sales_data)
```

#	Date	Product	Region	Revenue	Quantity
# 0	2023-01-01	Laptop	North	1000	5
# 1	2023-01-01	Tablet	North	500	10
# 2	2023-01-01	Laptop	South	1200	6
# 3	2023-01-01	Phone	North	300	15
# 4	2023-01-02	Tablet	South	550	11
# 5	2023-01-02	Laptop	North	1100	7
# 6	2023-01-02	Phone	South	350	18

Example 4: Basic pivot

```
print("\nA. Basic pivot: Revenue by Date and Product")
pivot1 = sales_data.pivot_table(
    index='Date',
    columns='Product',
    values='Revenue',
    aggfunc='sum' # Sum duplicates (Laptop appears twice on 2023-01-01)
)
print(pivot1)
```

# Product	Laptop	Phone	Tablet
# Date			
# 2023-01-01	2200	300	500
# 2023-01-02	1100	350	550

Example 4: Pivot - Multiple Aggregation Functions

```
print("\nB. Multiple aggregation functions:")
pivot2 = sales_data.pivot_table(
    index='Date',
    columns='Product',
    values='Revenue',
    aggfunc=['sum', 'mean', 'count']
)
print(pivot2)
```

	sum			mean			count		
Product	Laptop	Phone	Tablet	Laptop	Phone	Tablet	Laptop	Phone	Tablet
2023-01-01	2200	300	500	1100	300	500	2	1	1
2023-01-02	1100	350	550	1100	350	550	1	1	1

Example 4: Pivot - Multiple Value Columns

```
print("\nC. Multiple value columns:")
pivot3 = sales_data.pivot_table(
    index='Date',
    columns='Product',
    values=['Revenue', 'Quantity'],
    aggfunc='sum'
)
print(pivot3)
```

#		Revenue			Quantity		
#	Product	Laptop	Phone	Tablet	Laptop	Phone	Tablet
#	Date						
#	2023-01-01	2200	300	500	11	15	10
#	2023-01-02	1100	350	550	7	18	11

Example 4: Pivot - Multiple Level Index

```
print("\nD. Multi-level index (hierarchical rows):")
pivot4 = sales_data.pivot_table(
    index=['Date', 'Region'], # Two-level index
    columns='Product',
    values='Revenue',
    aggfunc='sum',
    fill_value=0 # Fill NaN with 0
)
```

```
print(pivot4)
```

#		Laptop	Phone	Tablet
# Date	Region			
# 2023-01-01	North	1000	300	500
#	South	1200	0	0
# 2023-01-02	North	1100	0	0
#	South	0	350	550

Example 4: Pivot - Using Margins (Grand totals)

```
print("\nE. With margins (Grand totals):")
pivot5 = sales_data.pivot_table(
    index='Region',
    columns='Product',
    values='Revenue',
    aggfunc='sum',
    margins=True,
    margins_name='Total'
)
```

```
print(pivot5)
```

# Product	Laptop	Phone	Tablet	Total
# Region				
# North	2100	300	500	2900
# South	1200	350	550	2100
# Total	3300	650	1050	5000

margins=True

- adds summary **total** rows and columns to your pivot table.

Example 5: Melt - Original DataSet

```
wide_sales = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar'],
    'NY_Sales': [10000, 12000, 11000],
    'LA_Sales': [8000, 8500, 8200],
    'CHI_Sales': [9000, 9500, 9200],
    'NY_Expenses': [2000, 2200, 2100],
    'LA_Expenses': [1500, 1600, 1550],
    'CHI_Expenses': [1800, 1900, 1850]
})

print("WIDE Format (Spreadsheet style):")
print("Each metric has separate columns for each city:")
print(wide_sales)
```

#	Month	NY_Sales	LA_Sales	CHI_Sales	NY_Expenses	LA_Expenses	CHI_Expenses
# 0	Jan	10000	8000	9000	2000	1500	1800
# 1	Feb	12000	8500	9500	2200	1600	1900
# 2	Mar	11000	8200	9200	2100	1550	1850

Example 5: Melt - Basic - Unpivot all value columns

```
print("\nA. Basic melt - Unpivot all value columns:")
melted1 = wide_sales.melt(
    id_vars='Month',          # Keep this column as identifier
    var_name='Metric_City',   # Name for the new column containing old column names
    value_name='Amount'       # Name for the new column containing values
)
print(melted1.head(10)) # Show first 10 rows
```

#	Month	Metric_City	Amount
# 0	Jan	NY_Sales	10000
# 1	Feb	NY_Sales	12000
# 2	Mar	NY_Sales	11000
# 3	Jan	LA_Sales	8000
# 4	Feb	LA_Sales	8500
# 5	Mar	LA_Sales	8200
# 6	Jan	CHI_Sales	9000
# 7	Feb	CHI_Sales	9500
# 8	Mar	CHI_Sales	9200
# 9	Jan	NY_Expenses	2000

Example 5: Melt - Selective

```
print("\nB. Selective melting - Specify which columns to unpivot:")
melted2 = wide_sales.melt(
    id_vars='Month',
    value_vars=['NY_Sales', 'LA_Sales', 'CHI_Sales'], # Only melt these
    var_name='City',
    value_name='Sales_Amount'
)
print(melted2)
```

#	Month	City	Sales_Amount
# 0	Jan	NY_Sales	10000
# 1	Feb	NY_Sales	12000
# 2	Mar	NY_Sales	11000
# 3	Jan	LA_Sales	8000
# 4	Feb	LA_Sales	8500
# 5	Mar	LA_Sales	8200
# 6	Jan	CHI_Sales	9000
# 7	Feb	CHI_Sales	9500
# 8	Mar	CHI_Sales	9200

Example 6: Real-world Example: Sales Analysis

```
# Create quarterly sales data in LONG format
quarterly_long = pd.DataFrame({
    'Year': [2022, 2022, 2022, 2022, 2023, 2023, 2023, 2023],
    'Quarter': ['Q1', 'Q2', 'Q3', 'Q4', 'Q1', 'Q2', 'Q3', 'Q4'],
    'Product_A': [100, 120, 130, 150, 110, 125, 140, 160],
    'Product_B': [80, 90, 95, 110, 85, 92, 100, 115],
    'Product_C': [50, 60, 70, 80, 55, 65, 75, 85]
})

print("Quarterly Sales (Semi-wide format):")
print(quarterly_long)
```

#	Year	Quarter	Product_A	Product_B	Product_C
# 0	2022	Q1	100	80	50
# 1	2022	Q2	120	90	60
# 2	2022	Q3	130	95	70
# 3	2022	Q4	150	110	80
# 4	2023	Q1	110	85	55
# 5	2023	Q2	125	92	65
# 6	2023	Q3	140	100	75
# 7	2023	Q4	160	115	85

Example 6: Melt to True Long Format

```
quarterly_melted = quarterly_long.melt(  
    id_vars=['Year', 'Quarter'],  
    value_vars=['Product_A', 'Product_B', 'Product_C'],  
    var_name='Product',  
    value_name='Sales'  
)  
print(quarterly_melted.head(12))
```

#	Year	Quarter	Product	Sales
# 0	2022	Q1	Product_A	100
# 1	2022	Q2	Product_A	120
# 2	2022	Q3	Product_A	130
# 3	2022	Q4	Product_A	150
# 4	2023	Q1	Product_A	110
# 5	2023	Q2	Product_A	125
# 6	2023	Q3	Product_A	140
# 7	2023	Q4	Product_A	160
# 8	2022	Q1	Product_B	80
# 9	2022	Q2	Product_B	90
# 10	2022	Q3	Product_B	95
# 11	2022	Q4	Product_B	110

Example 6: Pivot back

```
quarterly_pivot = quarterly_melted.pivot_table(  
    index=['Year', 'Quarter'],  
    columns='Product',  
    values='Sales',  
    aggfunc='sum'  
)
```

```
print(quarterly_pivot)
```

#		Product_A	Product_B	Product_C
#	Year Quarter			
#	2022 Q1	100	80	50
#	Q2	120	90	60
#	Q3	130	95	70
#	Q4	150	110	80
#	2023 Q1	110	85	55
#	Q2	125	92	65
#	Q3	140	100	75
#	Q4	160	115	85

Merging & Joining

❑ Concatenation (Stacking DataFrames)

```
pd.concat([df1, df2])
```

- **Used to stack DataFrames either:**
 - **Vertically (axis=0)** → add more rows
 - **Horizontally (axis=1)** → add more columns
- **Simple “append-like” behavior (no key matching).**
- **Example uses:**
 - Combining monthly files
 - Adding features from separate tables
 - Appending new batches of data

Merging (Database-Style Joins)

❑ `pd.merge(left, right, on='key', how='inner')` Works just like SQL joins.

❑ **Common how types:**

➤ **INNER JOIN**

- Only keys that appear in both tables.
- Intersection of keys.

➤ **LEFT JOIN**

- Keep all keys from the left DataFrame.
- Matching rows from the right; missing become **NaN**.

➤ **OUTER JOIN**

- Keep all keys from both tables.
- Union of keys (missing values filled with **NaN**).

Key Alignment Rules

- ❑ Merging uses the specified key column(s).
- ❑ If keys are duplicated in both tables, pandas produces:
 - A Cartesian product (all combinations of matching rows)
- ❑ **Example:**
 - Left table has 2 rows with key "A",
 - Right table has 3 rows with key "A",
 - **Joined result will have 6 rows for key A.**

“pandas produces a Cartesian product” Mean ?

❑ A Cartesian product happens when:

- The left table has multiple rows with the same key
- The right table has multiple rows with the same key
- During a merge, pandas matches every left row with every right row for that key

❑ So if:

- The **left** table has **L** rows with key "**A**"
- The **right** table has **R** rows with key "**A**"
- Then the merged result will have:

$L \times R$ rows for key "A"

❑ This is standard behavior in relational databases (SQL does the same).

Small Example (Very Easy to See)

- ❑ **Left table (has 2 rows for A.)**

Key	Value_Left
A	1
A	2

- ❑ **Right table (has 3 rows for A.)**

Key	Value_Right
A	100
A	200
A	300

- ❑ **Result (Cartesian product):**

Key	Value_Left	Value_Right
A	1	100
A	1	200
A	1	300
A	2	100
A	2	200
A	2	300

- ❑ **Number of rows = $2 \times 3 = 6$**
- ❑ **That's the Cartesian product.**

```
pd.merge(left, right, on='Key', how='inner')
```

Why does Pandas do this?

- ❑ Because pandas (like SQL) is following the rule:
 - **“Join on every row where the keys match.”**
- ❑ If a key appears multiple times on both sides, all combinations are considered valid matches.
- ❑ This is exactly how relational algebra defines a join.

Real-world example

❑ Imagine:

- A customer appears twice in the orders table (two purchases)
- A customer appears three times in the support tickets table

❑ If you join them on CustomerID, pandas will create:

- **2 (orders) × 3 (tickets) = 6 rows**

❑ **Because it pairs each order with each ticket for that customer.**

❑ **If you want only one row per key, you must:**

- **remove duplicates**
- **aggregate first**
- **or merge on a unique key**

Example 1 : Original DataFrames

```
# Create sample DataFrames
df1 = pd.DataFrame({
    'ID': ['A001', 'A002', 'A003'],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Dept': ['HR', 'Engineering', 'Marketing']
})

df2 = pd.DataFrame({
    'ID': ['A004', 'A005'],
    'Name': ['David', 'Eve'],
    'Dept': ['Sales', 'Engineering']
})

df3 = pd.DataFrame({
    'ID': ['A001', 'A002', 'A003'],
    'Salary': [75000, 95000, 65000],
    'Start_Date': ['2020-01-15', '2019-03-20', '2021-06-10']
})

print(df1)

#      ID      Name      Dept
# 0  A001    Alice      HR
# 1  A002     Bob  Engineering
# 2  A003  Charlie  Marketing

print("\nDataFrame 2 (Employees 4-5):")
print(df2)

#      ID      Name      Dept
# 0  A004    David      Sales
# 2  A005     Eve  Engineering

print("\nDataFrame 3 (Salary Info):")
print(df3)

#      ID  Salary  Start_Date
# 0  A001   75000  2020-01-15
# 1  A002   95000  2019-03-20
# 2  A003   65000  2021-06-10
```

Example 1 : Concatenation

```
print("A. VERTICAL CONCATENATION (axis=0) - Appending rows:")
vertical_concat = pd.concat([df1, df2], axis=0, ignore_index=True)
print("Append df2 below df1:")
print(vertical_concat)
#      ID      Name      Dept
# 0  A001    Alice        HR
# 1  A002     Bob  Engineering
# 2  A003  Charlie  Marketing
# 3  A004    David        Sales
# 4  A005     Eve  Engineering

print("\nB. HORIZONTAL CONCATENATION (axis=1):")
horizontal_concat = pd.concat([df1, df3[['Salary', 'Start_Date']]], axis=1)
print("Join df1 and df3 side-by-side:")
print(horizontal_concat)
#      ID      Name      Dept  Salary  Start_Date
# 0  A001    Alice        HR   75000  2020-01-15
# 1  A002     Bob  Engineering   95000  2019-03-20
# 2  A003  Charlie  Marketing   65000  2021-06-10
```

- If **ignore_index = False** original indexes are preserved
- If **ignore_index = True** pandas creates a brand-new index: 0, 1, 2, 3,
- ❑ Simple Rule of Thumb
 - If you're building a new dataset → use **ignore_index=True**
 - If you're combining data that relies on index labels → **keep default (False)**

Example 2 : Original DataFrames

```
employees = pd.DataFrame({
    'Emp_ID': ['E001', 'E002', 'E003', 'E004', 'E005'],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Dept_ID': ['D01', 'D02', 'D01', 'D03', 'D02']
})
```

```
departments = pd.DataFrame({
    'Dept_ID': ['D01', 'D02', 'D03', 'D04'],
    'Dept_Name': ['HR', 'Engineering', 'Marketing', 'Sales'],
    'Manager': ['Carol', 'Frank', 'Grace', 'Henry']
})
```

```
salaries = pd.DataFrame({
    'Emp_ID': ['E001', 'E002', 'E003', 'E006'], # Note: E006 doesn't exist in employees
    'Salary': [75000, 95000, 65000, 80000],
    'Bonus': [5000, 10000, 3000, 6000]
})
```

```
print(employees)
#   Emp_ID   Name Dept_ID
# 0  E001   Alice   D01
# 1  E002    Bob   D02
# 2  E003  Charlie   D01
# 3  E004   David   D03
# 4  E005    Eve   D02
```

```
print("\nDepartments DataFrame:")
print(departments)
#   Dept_ID Dept_Name Manager
# 0    D01         HR   Carol
# 1    D02  Engineering   Frank
# 2    D03   Marketing   Grace
# 3    D04         Sales   Henry
```

```
print("\nSalaries DataFrame:")
print(salaries)
#   Emp_ID  Salary  Bonus
# 0  E001   75000   5000
# 1  E002   95000  10000
# 2  E003   65000   3000
# 3  E006   80000   6000
```

Example 2 : Inner Join (default)

```
inner_join = pd.merge(employees, departments, on='Dept_ID', how='inner')
print("Employees w Departments (inner join):")
print(inner_join)
```

#	Emp_ID	Name	Dept_ID	Dept_Name	Manager
# 0	E001	Alice	D01	HR	Carol
# 1	E003	Charlie	D01	HR	Carol
# 2	E002	Bob	D02	Engineering	Frank
# 3	E005	Eve	D02	Engineering	Frank
# 4	E004	David	D03	Marketing	Grace

```
print(employees)
```

#	Emp_ID	Name	Dept_ID
# 0	E001	Alice	D01
# 1	E002	Bob	D02
# 2	E003	Charlie	D01
# 3	E004	David	D03
# 4	E005	Eve	D02

```
print("\nDepartments DataFrame:")
```

```
print(departments)
```

#	Dept_ID	Dept_Name	Manager
# 0	D01	HR	Carol
# 1	D02	Engineering	Frank
# 2	D03	Marketing	Grace
# 3	D04	Sales	Henry

```
print("\nSalaries DataFrame:")
```

```
print(salaries)
```

#	Emp_ID	Salary	Bonus
# 0	E001	75000	5000
# 1	E002	95000	10000
# 2	E003	65000	3000
# 3	E006	80000	6000

Example 2 : Left Join (default)

```
left_join = pd.merge(employees, departments, on='Dept_ID', how='left')
print("Employees ⋈ Departments (left join):")
print(left_join)
```

#	Emp_ID	Name	Dept_ID	Dept_Name	Manager
# 0	E001	Alice	D01	HR	Carol
# 1	E002	Bob	D02	Engineering	Frank
# 2	E003	Charlie	D01	HR	Carol
# 3	E004	David	D03	Marketing	Grace
# 4	E005	Eve	D02	Engineering	Frank

```
print(employees)
#   Emp_ID  Name Dept_ID
# 0  E001   Alice   D01
# 1  E002    Bob   D02
# 2  E003  Charlie   D01
# 3  E004   David   D03
# 4  E005    Eve   D02

print("\nDepartments DataFrame:")
print(departments)
#   Dept_ID  Dept_Name Manager
# 0    D01           HR   Carol
# 1    D02  Engineering   Frank
# 2    D03  Marketing   Grace
# 3    D04          Sales   Henry

print("\nSalaries DataFrame:")
print(salaries)
#   Emp_ID  Salary  Bonus
# 0  E001   75000   5000
# 1  E002   95000  10000
# 2  E003   65000   3000
# 3  E006   80000   6000
```

Example 2 : Right Join (default)

```
right_join = pd.merge(employees, departments, on='Dept_ID', how='right')
print("Employees ⋈ Departments (right join):")
print(right_join)
```

#	Emp_ID	Name	Dept_ID	Dept_Name	Manager
# 0	E001	Alice	D01	HR	Carol
# 1	E003	Charlie	D01	HR	Carol
# 2	E002	Bob	D02	Engineering	Frank
# 3	E005	Eve	D02	Engineering	Frank
# 4	E004	David	D03	Marketing	Grace
# 5	NaN	NaN	D04	Sales	Henry

```
print(employees)
```

#	Emp_ID	Name	Dept_ID
# 0	E001	Alice	D01
# 1	E002	Bob	D02
# 2	E003	Charlie	D01
# 3	E004	David	D03
# 4	E005	Eve	D02

```
print("\nDepartments DataFrame:")
```

```
print(departments)
```

#	Dept_ID	Dept_Name	Manager
# 0	D01	HR	Carol
# 1	D02	Engineering	Frank
# 2	D03	Marketing	Grace
# 3	D04	Sales	Henry

```
print("\nSalaries DataFrame:")
```

```
print(salaries)
```

#	Emp_ID	Salary	Bonus
# 0	E001	75000	5000
# 1	E002	95000	10000
# 2	E003	65000	3000
# 3	E006	80000	6000

Example 2 : Outer Join (default)

```
outer_join = pd.merge(employees, departments, on='Dept_ID', how='outer')
print("Employees & Departments (full outer join):")
print(outer_join)
```

#	Emp_ID	Name	Dept_ID	Dept_Name	Manager
# 0	E001	Alice	D01	HR	Carol
# 1	E003	Charlie	D01	HR	Carol
# 2	E002	Bob	D02	Engineering	Frank
# 3	E005	Eve	D02	Engineering	Frank
# 4	E004	David	D03	Marketing	Grace
# 5	NaN	NaN	D04	Sales	Henry

```
print(employees)
```

#	Emp_ID	Name	Dept_ID
# 0	E001	Alice	D01
# 1	E002	Bob	D02
# 2	E003	Charlie	D01
# 3	E004	David	D03
# 4	E005	Eve	D02

```
print("\nDepartments DataFrame:")
```

```
print(departments)
```

#	Dept_ID	Dept_Name	Manager
# 0	D01	HR	Carol
# 1	D02	Engineering	Frank
# 2	D03	Marketing	Grace
# 3	D04	Sales	Henry

```
print("\nSalaries DataFrame:")
```

```
print(salaries)
```

#	Emp_ID	Salary	Bonus
# 0	E001	75000	5000
# 1	E002	95000	10000
# 2	E003	65000	3000
# 3	E006	80000	6000

Example 2 : Merging employees with salaries (Inner)

```
emp_salary = pd.merge(employees, salaries, on='Emp_ID', how='inner')
print("Employees ⋈ Salaries:")
print(emp_salary)
```

#	Emp_ID	Name	Dept_ID	Salary	Bonus
# 0	E001	Alice	D01	75000	5000
# 1	E002	Bob	D02	95000	10000
# 2	E003	Charlie	D01	65000	3000

```
print(employees)
```

#	Emp_ID	Name	Dept_ID
# 0	E001	Alice	D01
# 1	E002	Bob	D02
# 2	E003	Charlie	D01
# 3	E004	David	D03
# 4	E005	Eve	D02

```
print("\nDepartments DataFrame:")
print(departments)
```

#	Dept_ID	Dept_Name	Manager
# 0	D01	HR	Carol
# 1	D02	Engineering	Frank
# 2	D03	Marketing	Grace
# 3	D04	Sales	Henry

```
print("\nSalaries DataFrame:")
print(salaries)
```

#	Emp_ID	Salary	Bonus
# 0	E001	75000	5000
# 1	E002	95000	10000
# 2	E003	65000	3000
# 3	E006	80000	6000

Example 2 : Merging employees with salaries (Left)

```
emp_salary_left = pd.merge(employees, salaries, on='Emp_ID', how='left')
print("Employees & Salaries:")
print(emp_salary_left)
```

#	Emp_ID	Name	Dept_ID	Salary	Bonus
# 0	E001	Alice	D01	75000.0	5000.0
# 1	E002	Bob	D02	95000.0	10000.0
# 2	E003	Charlie	D01	65000.0	3000.0
# 3	E004	David	D03	NaN	NaN
# 4	E005	Eve	D02	NaN	NaN

```
print(employees)
```

#	Emp_ID	Name	Dept_ID
# 0	E001	Alice	D01
# 1	E002	Bob	D02
# 2	E003	Charlie	D01
# 3	E004	David	D03
# 4	E005	Eve	D02

```
print("\nDepartments DataFrame:")
```

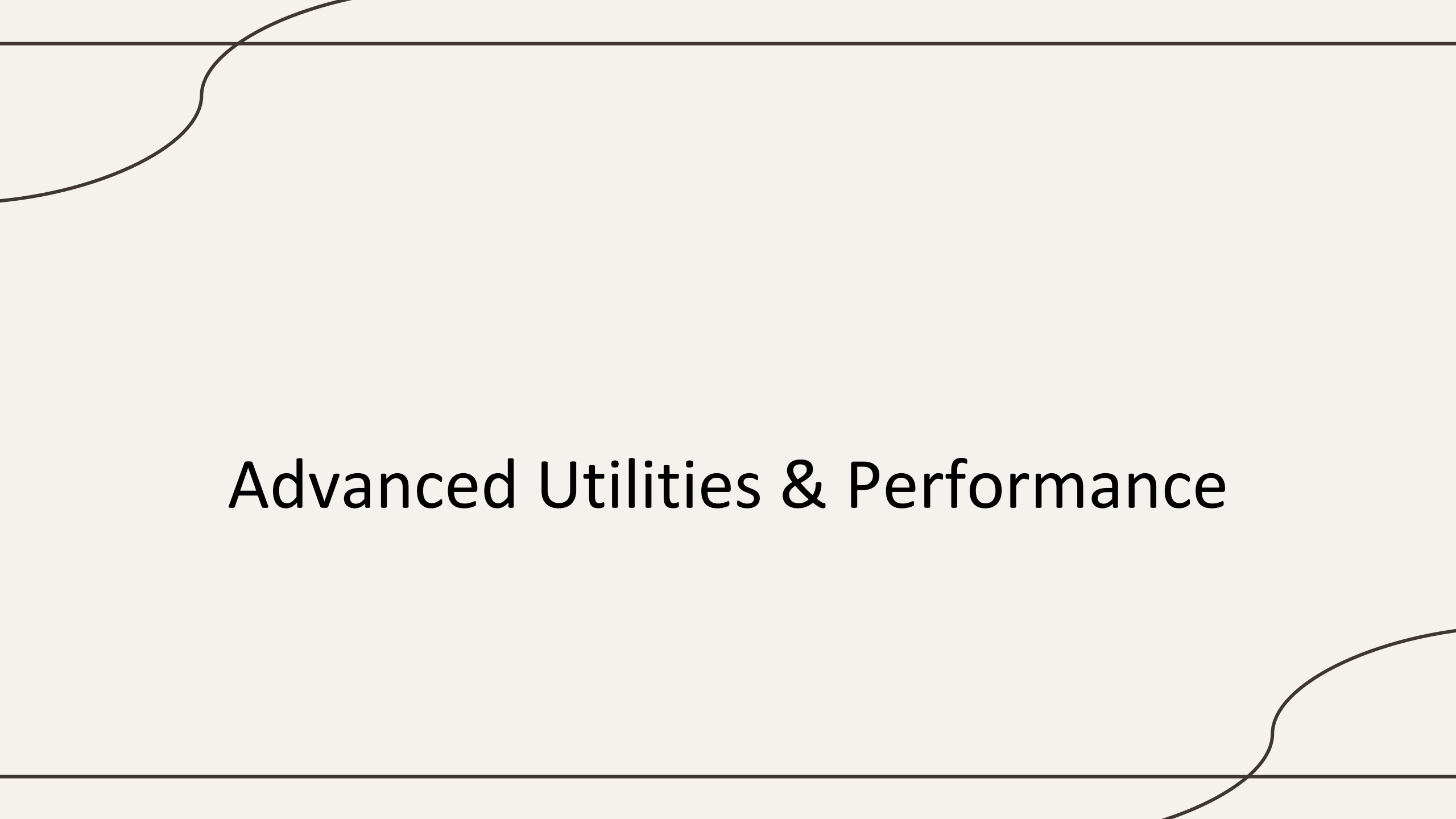
```
print(departments)
```

#	Dept_ID	Dept_Name	Manager
# 0	D01	HR	Carol
# 1	D02	Engineering	Frank
# 2	D03	Marketing	Grace
# 3	D04	Sales	Henry

```
print("\nSalaries DataFrame:")
```

```
print(salaries)
```

#	Emp_ID	Salary	Bonus
# 0	E001	75000	5000
# 1	E002	95000	10000
# 2	E003	65000	3000
# 3	E006	80000	6000

The image features two horizontal lines, one at the top and one at the bottom. Each line has a curved, organic shape extending from its left and right ends, resembling a stylized wave or a decorative flourish. The text is centered between these two lines.

Advanced Utilities & Performance

Time Series Analysis

□ Datetime Index

- Convert a column to datetime using `pd.to_datetime()`.
- Setting it as the index enables powerful time-based operations.

- **Example:**

```
df['Date'] = pd.to_datetime(df['Date'])  
df = df.set_index('Date')
```

- **Benefits**

- Label-based slicing by time
- Partial string indexing

```
df['2023-01']      # All data in January 2023  
df['2023']         # Entire year
```

Time Series Analysis (Cont.)

❑ Resampling (.resample())

Change the frequency of time-series data.

➤ Downsampling (Higher → Lower frequency)

Examples:

- Minutes → Hours
- Hours → Days

```
df.resample('H').mean()    # minute → hour
```

Requires aggregation, like mean, sum:

➤ Upsampling (Lower → Higher frequency)

Examples:

- Hourly → Minutes

```
df.resample('T').ffill()   # hour → minute (fill forward)
```

Requires filling, like:

After upsampling, pandas creates new time rows that have no data yet

.ffill() fills these gaps using the nearest previous valid entry.

Example 1 – Original DataFrame

```
data = {  
    'date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05',  
            '2023-02-01', '2023-02-02', '2023-02-03', '2023-02-04', '2023-02-05',  
            '2023-03-01', '2023-03-02', '2023-03-03', '2023-03-04', '2023-03-05'],  
    'sales': [100, 120, 110, 130, 125,  
             150, 160, 155, 170, 165,  
             200, 210, 205, 220, 215],  
    'temperature': [32, 34, 33, 35, 36,  
                   38, 40, 39, 41, 42,  
                   45, 46, 47, 48, 49]  
}
```

```
df = pd.DataFrame(data)  
print("Original DataFrame (string dates):")  
print(df.head())
```

#	date	sales	temperature
# 0	2023-01-01	100	32
# 1	2023-01-02	120	34
# 2	2023-01-03	110	33
# 3	2023-01-04	130	35
# 4	2023-01-05	125	36

- df.head() returns the first **5** rows of a DataFrame.
- It is used to quickly preview your data.

Example 1 – Conversion to Datetime

```
print(f"\ndf['date'].dtype: {df['date'].dtype}") # object (string)

print("\nA. Convert to datetime:")
df['date'] = pd.to_datetime(df['date'])
print(f"After conversion: {df['date'].dtype}") # datetime64[ns]

print("\nB. Set as index:")
df_time = df.set_index('date')
print("DataFrame with DatetimeIndex:")
print(df_time.head())
```

#	sales	temperature
# date		
# 2023-01-01	100	32
# 2023-01-02	120	34
# 2023-01-03	110	33
# 2023-01-04	130	35
# 2023-01-05	125	36

Example 1 – Get January 2023

```
data = {  
    'date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05',  
            '2023-02-01', '2023-02-02', '2023-02-03', '2023-02-04', '2023-02-05',  
            '2023-03-01', '2023-03-02', '2023-03-03', '2023-03-04', '2023-03-05'],  
    'sales': [100, 120, 110, 130, 125,  
              150, 160, 155, 170, 165,  
              200, 210, 205, 220, 215],  
    'temperature': [32, 34, 33, 35, 36,  
                    38, 40, 39, 41, 42,  
                    45, 46, 47, 48, 49]  
}  
  
jan_data = df_time['2023-01']  
print(jan_data)
```

#	date	sales	temperature
#	2023-01-01	100	32
#	2023-01-02	120	34
#	2023-01-03	110	33
#	2023-01-04	130	35
#	2023-01-05	125	36

Example 1 – Get data between dates

```
data = {  
    'date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-05',  
            '2023-02-01', '2023-02-02', '2023-02-03', '2023-02-04', '2023-02-05',  
            '2023-03-01', '2023-03-02', '2023-03-03', '2023-03-04', '2023-03-05'],  
    'sales': [100, 120, 110, 130, 125,  
              150, 160, 155, 170, 165,  
              200, 210, 205, 220, 215],  
    'temperature': [32, 34, 33, 35, 36,  
                    38, 40, 39, 41, 42,  
                    45, 46, 47, 48, 49]  
}
```

```
date_range = df_time['2023-01-02':'2023-01-04']  
print(date_range)
```

	sales	temperature
# date		
# 2023-01-02	120	34
# 2023-01-03	110	33
# 2023-01-04	130	35

Example 2 – Original DataFrame

```
dates = pd.date_range("2024-01-01", periods=8, freq="D")
values = [10, 12, 11, 15, 14, 18, 17, 16]

df = pd.DataFrame({"value": values}, index=dates)

print(df)
```

#	value
# 2024-01-01	10
# 2024-01-02	12
# 2024-01-03	11
# 2024-01-04	15
# 2024-01-05	14
# 2024-01-06	18
# 2024-01-07	17
# 2024-01-08	16

Example 2 – Downsampling

```
#  
# 2) Downsample daily -> 2-day mean  
#     (groups every 2 days and computes average)  
# -----  
mean_2d = df.resample("2D").mean()  
  
print(mean_2d)  
#           value  
# 2024-01-01    11.0  
# 2024-01-03    13.0  
# 2024-01-05    16.0  
# 2024-01-07    16.5  
  
# -----  
# 3) Downsample daily -> weekly sum  
#     W-SUN means "weekly, ending on Sunday"  
# -----  
sum_w = df.resample("W-SUN").sum()  
  
print(sum_w)  
#           value  
# 2024-01-07      97  
# 2024-01-14      16
```

Example 2 – Upsampling

```
# UPSAMPLING
# =====

# -----
# 4) Upsample daily -> 12-hour frequency with forward-fill
#     Creates new timestamps and fills gaps using last known value
# -----
ups_12h_ffill = df.resample("12H").ffill()

print(ups_12h_ffill.head(10))
```

#	value
# 2024-01-01 00:00:00	10
# 2024-01-01 12:00:00	10
# 2024-01-02 00:00:00	12
# 2024-01-02 12:00:00	12
# 2024-01-03 00:00:00	11
# 2024-01-03 12:00:00	11
# 2024-01-04 00:00:00	15
# 2024-01-04 12:00:00	15
# 2024-01-05 00:00:00	14
# 2024-01-05 12:00:00	14

Example 2 – Upsampling (Cont.)

```
# 5) Upsample daily -> 6-hour frequency with time interpolation
#   Creates intermediate points using time-based linear interpolation
#   -----
ups_6h_interp = df.resample("6H").interpolate("time")

print(ups_6h_interp.head(12))
#           value
# 2024-01-01 00:00:00  10.00
# 2024-01-01 06:00:00  10.50
# 2024-01-01 12:00:00  11.00
# 2024-01-01 18:00:00  11.50
# 2024-01-02 00:00:00  12.00
# 2024-01-02 06:00:00  11.75
# 2024-01-02 12:00:00  11.50
# 2024-01-02 18:00:00  11.25
# 2024-01-03 00:00:00  11.00
# 2024-01-03 06:00:00  12.00
# 2024-01-03 12:00:00  13.00
# 2024-01-03 18:00:00  14.00
..
```

1. Creates new timestamps every **6 hours**.
2. Inserts **NaNs** for the new timestamps **between** known daily points.
3. Fills those **NaNs** using linear interpolation based on time.

So if you have:

- 2024-01-01 00:00 = 10
- 2024-01-02 00:00 = 12
- Then at **6** hours after midnight ($\frac{1}{4}$ of a day), "time" gives:

$$10 + (12 - 10) * 0.25 = 10.5$$

Categorical Data (Memory Optimization)

❑ The Problem

- Repeated string values consume excessive memory because:
- Each string is a Python object
- Stored repeatedly in memory
- Example: "Male", "Female", "Male", "Male", ...

❑ The Solution: `astype('category')`

```
df[ 'Gender' ] = df[ 'Gender' ].astype( 'category' )
```

Categorical Data (Memory Optimization)

❑ How it works

- pandas stores unique values once
- Maps them internally to integer codes (0, 1, ...)

❑ Benefits

- Massive memory reduction (often 5 – 10x)
- **Faster:**
 - `groupby()`
 - Sorting
 - Filtering

❑ **Use categorical dtype for low-cardinality string columns.**

Performance & Best Practices

Avoid Explicit Loops

Use loops **only** when absolutely unavoidable.

❑ **Bad:** loop over rows with iterrows()

```
# BAD: row-by-row Python Loop
sum_list = []
for idx, row in df.iterrows():
    sum_list.append(row["A"] + row["B"])

df["Sum_loop"] = sum_list

print(df[["A", "B", "Sum_loop"]])
#   A  B  Sum_loop
# 0  1  10         11
# 1  2  20         22
# 2  3  30         33
# 3  4  40         44
# 4  5  50         55
```

❑ **Good:** vectorized column arithmetic

```
df["Sum_vec"] = df["A"] + df["B"]

print(df[["A", "B", "Sum_vec"]])
#   A  B  Sum_vec
# 0  1  10         11
# 1  2  20         22
# 2  3  30         33
# 3  4  40         44
# 4  5  50         55
```

- ❑ **Vectorized ops run in optimized C/NumPy loops internally, instead of Python looping row-by-row.**

Performance & Best Practices

Prefer Vectorization

- ❑ **Bad:** apply row-wise (Python function called for every row)

```
df["Sum_apply"] = df.apply(lambda x: x["A"] + x["B"], axis=1)

print(df[["A", "B", "Sum_apply"]])
```

#	A	B	Sum_apply
# 0	1	10	11
# 1	2	20	22
# 2	3	30	33
# 3	4	40	44
# 4	5	50	55

- ❑ **Good:** direct vectorized operation

```
df["Sum_good"] = df["A"] + df["B"]

print(df[["A", "B", "Sum_good"]])
```

#	A	B	Sum_good
# 0	1	10	11
# 1	2	20	22
# 2	3	30	33
# 3	4	40	44
# 4	5	50	55

eval()

eval(): fast column arithmetic / fewer temporaries

Example: compute a derived column with eval

```
df["Expr_eval"] = df.eval("A + 2*B - C")

print(df[["A", "B", "C", "Expr_eval"]])
```

#	A	B	C	Expr_eval
# 0	1	10	2	19
# 1	2	20	0	42
# 2	3	30	4	59
# 3	4	40	8	76
# 4	5	50	1	104

Same thing without eval (also correct, but may allocate intermediates)

```
df["Expr_plain"] = df["A"] + 2*df["B"] - df["C"]

print(df[["Expr_plain"]])
```

#	Expr_plain
# 0	19
# 1	42
# 2	59
# 3	76
# 4	104

When does eval() help?

When expressions get longer (many columns/ops) and the DataFrame is big, eval() can reduce temporary arrays and use **NumExpr** for speed.

query()

query(): fast filtering with expression strings.

Example: filter rows using query

```
filtered = df.query("group == 'x' and B >= 20")

print(filtered)
```

#	A	B	C	group	Sum_loop	Sum_vec	Sum_apply
	Expr_eval		Expr_plain				
# 1	2	20	0	x	22	22	22
	42		42				
# 4	5	50	1	x	55	55	55
	104		104				

Equivalent boolean indexing (also correct)

```
filtered2 = df[(df["group"] == "x") & (df["B"] >= 20)]

print(filtered2[["A", "B", "C", "group"]])
```

#	A	B	C	group
# 1	2	20	0	x
# 4	5	50	1	x